CS4 Final Year Project:

The Ultimate RISC

Stephen A. Loughran

Supervisor: Nigel Topham

November 27, 2017

Abstract

Computer Design is a highly competitive field, where there is much interest in the possibility of designing high performance computers quickly, cheaply and reliably. The cost and performance requirements may be satisfied by RISC architectures, and the application of Formal Methods to hardware design promises new levels of quality. This report is a description of a final year project, the design, formal specification and implementation of the Ultimate RISC, a single instruction computer.

The project was a demonstration that such a computer is simple enough to be designed and built within a single year, although the implementation does suffer from some limitations.

The concept of a single instruction computer is discussed in general, concluding that whilst it is a fast and compact design which could be useful in some applications the memory bandwidth it requires limits its performance.

Contents

Chapter 1

Introduction

Research is continuously under way to try and build the fastest computers with the technology available. Low cost and high reliability are often of as high a priority as performance. A recent concept is that of RISC computers, which increase performance by reducing the number of instructions the computer can understand.

The aim of this project was to take this architecture to its natural conclusion, to build a computer capable of executing only a single instruction. It has demonstrated that such a machine can be easily and cheaply built, yet may be compared, in terms of raw computing power, with commercial microprocessors.

The architecture of the computer was described mathematically, a technique known as *Formal Specification*, and is one of the few computers to have been so designed. This technique increases the likely reliability of a computer, but severely limits its complexity. The project therefore provides an example into the formal specification of computer systems, and a demonstration of the the

associated difficulties.

The computer implemented is a 32-bit processor with an integer ALU and 32 Kilobytes of memory, built out of simple MSI Integrated Circuits. It fits onto a single APM card, and can be connected to such a workstation via a co-processor. A monitor program has been written to control the Ultimate RISC, providing facilities for hardware and software development.

It should be capable of executing approximately 3.3 Million instructions every second, although this has not been achieved for a number of reasons.

1.1 The Reduced Instruction Set Computer

> *"A good artist can create fine art with crayons or oil paints (or whatever). Assembly languages are definitely the crayons of the computer world. RISCs are kind of like the little box of Crayolas with 16 colors, a CISC, like the VAX, is like the big box with 64. Ever notice how it was that the black, red and blue crayons always ended up the smallest in that big box and the mauve crayon looked brand new?"*
>
> David L. Smith
> FPS Computing, San Diego

The concept of a Reduced Instruction Set Computer (RISC) was born in the late Nineteen-Seventies. Researchers saw that regardless of the number of instructions which a computer could execute, a small core of instructions accounted for the largest proportion of code. This was despite the fact that for years computer architects had been designing instruction sets especially to support high level

languages. These long, complex instructions reduced the size of programs, but as the cost of memories fell dramatically this fact became unimportant.

Supporting rarely used instructions extracted a price in both computer performance and complexity. Decoding and executing a large set of instructions used a very complex control unit, requiring space and a significant development effort. Over the years techniques such as microcoding and nanocoding were developed, which effectively stored a small set of routines within the CPU. These routines broke the larger instructions down into smaller instructions which were then executed. As the speed and size of external memories increased, the justification for such techniques disappeared.

Knowing all the complex instructions could be produced out of a larger number of simpler instructions, researchers proposed that computers should be made to execute the simple instructions alone —*a Reduced Instruction Set Computer*.

The basic tenets of RISC were:-

- A small number of fixed length instructions
- Hard-wired instruction decode —-no microcoding
- Load and Store instructions access memory —all others operate between registers
- Single cycle instruction execution
- An optimising compiler to take full advantage of the instruction set

The instructions chosen were those most used by compiler generated code, and a any others needed to support special hardware features.

Reducing the complexity of the control section, the benefits were:-

- reduced CPU size
- reduced design time

RISC computers could be designed and built quickly, taking advantage of the latest fabrication processes. The CPUs could be built smaller and cheaper, or the spare die area filled with simple repetitive structures:-

- a large register file
- a barrel shifter
- on chip RAM or caches
- wider and multiple busses
- inter-process and inter-processor communication links

These provided significant increases in performance, and yet were easy to design, making for highly competitive products.

The question which RISC architectures raise is simple -
"How many instructions are really needed? What is the Ultimate RISC?"
In fact a computer need only be capable of executing a single instruction.

1.2 Ultimate RISCs

In 1956 W. van der Poel showed how a single instruction was computationally sufficient for all programming needs. Describing the instruction set of an existing computer, he demonstrated by a process of elimination that only one instruction was actually needed [**?**]. The instruction was to subtract a value in memory

from an accumulator, rewriting the result back to the memory location. If the number subtracted was bigger than the accumulator then the next instruction was skipped. Bearing no resemblance to common instructions, this Reverse Subtract instruction would be inherently inefficient. This is because of the large number of instructions needed to perform any useful operation.

My project was originally intended to be an implementation of a computer to execute this instruction. However, I changed my plans on reading an article in the June 1988 edition of Computer Architecture News [**?**]. This article described how a computer could be built using an instruction to move the contents of one memory location to another address. Active elements within the computer's memory provide program control and mathematical functions. This 'MOVE' instruction was far more intuitive, and more efficient for a computer to support, so I decided to design and build such a computer. It offered the following advantages over the older design:-

- the article provided a much clearer description of the computer
- machine code programs should be easier to produce
- the performance of such a computer should be faster due to the ability to support operations other than subtraction.
- it is a flexible architecture, permitting much further expansion.

During the year research revealed the existence of three previous implementations of a single instruction computer. Two were based on the van der Poel instruction, while the third was MOVE based —even though it predated the article I used.

1. Negev University, 1976 [**?**]
   A MOVE based instruction with four index registers was implemented in an eight bit wide computer.

2. Manchester University, 1987
   A final year CS undergraduate built a computer based upon the reverse subtract instruction.

3. Philips Research labs, 1987 [**?**]
   A team of people implemented van der Poel's computer, with a PL/0 interpreter to execute pascal code. The performance of this was claimed to be significantly slower than a M68000 based system.

A major factor separating my design from these is the fact that the architecture was specified formally. This provides:-

- A specification for software developers to use.
- A simulation of the computer
- The ability to verify the hardware against the specification.

A fellow student has produced a compiler for the Ultimate RISC. This means the final product consists of a computer interfaced to a host, which can cross-compile Pascal programs onto my computer. The design process therefore included consultations with this other student, to enable the machine and the compiler to

work together. While more akin to how an actual computer would be produced than a hardware project alone, it meant all design decisions were the result of heated discussion.

1.3 Formal Specification of Hardware

There are a number of ways one can describe a computer. The English Language is useful for providing a brief and informal description to another person. A circuit diagram together with component data sheets tells someone with hardware knowledge exactly how to build one. Such a low level description does not describe how the system would appear to a programmer. Techniques have also been developed to describe the operation of control sections at the register transfer level. This is useful to microcode and control unit designers. Diagrams are also used at most levels to convey information.

There is no real correlation between all these views of a design. There is no way of proving that the hardware will do what the architecture states until it has actually been built. Of course, people are experienced in converting a design from one level to another, but there is always the possibility of their making a mistake. If an error is only detected when a prototype is built it will have wasted valuable time. If an error only surfaces later in a product's life cycle, when large numbers have been produced, then it could be very expensive to correct. With the trend towards embedding microprocessors within safety critical systems, any fault can be potentially disastrous. This has raised interest in applying formal specification techniques to hardware design.

Formal specifications mathematically describe a system at different levels of abstraction. At a very high level one defines entities such as 'memory', giving their basic properties but without stating how they are to be implemented. By relating such entities together, the architecture of a computer can be described. Top down design of the system is then performed by expanding the internal description of each entity. As long as the interfaces remain the same this can be done on a module by module basis. Each module can be subdivided until

specified as a collection of related components. This process can be continued until even the individual gates of a VLSI IC are described.

With a precise mathematical description one can prove that system will have certain desired properties. For example, a memory which stores different data at different locations, or the equivalence between the mathematics a computer performs and the functions required by international standards. The consistency between different specifications can also be proven. This is important as if one can show that the architecture and the gate level specifications are equivalent, and also that the architecture has certain properties, then one can infer that the gate level design behaves likewise. If this low level description can then be built, then it is highly likely that the finished product will also behave as specified.

What might formal methods do for hardware design?

- increase the likelihood of first time correctness
- increase confidence in the reliability of a system
- provide a baseline description for software developers to design their software around.

Given the cost and turnaround time of some hardware fabrication processes, time, money and effort is well spent proving the correctness of a design prior to construction.

What can't formal methods do yet?

- Make the design process faster.
  It is currently very slow to specify a system, since one must start from scratch describing components and operations. Proving the correctness

of any specification consumes large amounts of time of both people and computers.

- Accurately model electronic circuits.
  While digital circuits are normally viewed as communicating with binary data, no signal can exist in purely two states. There are problems such as crosstalk and fanout which cause a circuit to behave unreliably.

- Guarantee a system will always work.
  A specification must include clauses that it will only hold if certain preconditions —e.g. supply voltages and signal setup and hold times— are met. If these conditions are not satisfied then the system's behaviour will be nondeterministic.

When specifying VLSI designs, the process can be continued down to individual gates. These gates can be described and modelled; the operation of more complex structures inferred from them. It is more difficult describing how systems behave when building a system from larger components only described in data sheet form. One must respecify the data sheets in the notation used, and will always have to rely upon the correctness of the data sheet and the respecification. Certain equipment manufacturers are said to be looking at formal specification as a way of describing components to be ordered from subcontractors. This will be useful as rigorous product acceptance criteria. It may also mean that in the future component manufacturers will supply formal specifications of their products. This will be of use only if standard notations are used.

There have been to my knowledge two other microprocessors which have been formally specified [**?**]. The Viper-1 microprocessor was a government research project to produce a reliable microprocessor for military applications. Another microprocessor was specified in Cambridge, as an exercise in specification, and only later actually built. The former of these bears quite a resemblance to

my design, even though I had no knowledge of the Viper-1 when designing my computer.

The Viper-1 Microprocessor

Designed at RSRE in Malvern, the notation used was HOL, which originated in the Cambridge Computing Laboratory. With a goal of high reliability rather than performance and also due to the limitations of current specification methodologies, the processor is not much more complex than my own.

- 32 bit data bus
- 20 bit address bus (with a separate peripheral address space)
- four registers - **A**, **X**, **Y**, **P**
- 32 instructions—
  - 16 comparison instructions
  - 13 ALU operations
  - 2 data fetch operations
  - 1 program control

  The destination can be selected for any result, and includes a conditional write to the program counter.
- Neither interrupt nor stack mechanisms.
- Static RAM for extra reliability.

First specifying the processor at an architectural level, the design was reified down to the microstates of instruction execution. A manual proof of equivalence

took three weeks. A later automated proof took six person-months and found mistakes undetected manually. Finally an ELLA description of the Viper-1 was made and simulated against the predictions of the specification. This description was sent to different subcontractors to actually fabricate on silicon. By having multiple versions of the same processor there is less risk of fabrication dependent problems. Safety critical applications can then use multiple independently sourced microprocessors to verify all results. Multiple sourcing proved a good decision as one of the original microprocessors so fabricated did not work. There are reported claims that this was due to one of the subcontractors attempting to manually optimise the ELLA description, which negated all the previous attempts to ensure reliability.

From this actual example one can see that:-

- there is currently a tradeoff between reliability and high performance
- for a fully reliable design there should be no manual intervention, as this seems to encourage mistakes.
- the use of Formal Methods is a slow process.

Of course, one can not be sure how reliable automated proof and fabrication processes are, and thus absolute correctness can not be guaranteed. The correctness of computers designed using automated proof systems is still likely to be much better than those designed informally.

Chapter 2

Architecture

2.1 Instruction Set

The single instruction supported is

```
MOVE source,destination
```

The contents of the source address are moved to the destination. As it stands, any indirect addressing —such as pointers or array indexing— would have be performed by self-modifying code, which makes programs difficult to write and debug. By providing index registers for the operands these problems disappear. The price is increased hardware complexity and some delays caused by the addition of offsets.

There are two index registers, one for the source operand and another for the destination. This should be more flexible than a single index register. Using

one bit per operand to indicate whether it is to be indexed or not, two bits are needed per instruction to indicate the addressing modes.

It can be argued that these bits constitute an instruction opcode field and thus there are actually four instructions. An alternative viewpoint is that the computer has a 'virtual memory' for operands of double the real memory size. Virtual addresses with the most significant bit set are merely translated into the real address segment starting at the index register's value.

2.2 Block Structure

The block diagram of the computer is given in figure **??**. Note the absence of any instruction decode unit. The **Control Unit** generates the control and timing signals. It is controlled by a host computer via the **Host Interface**. The **Execution Unit** fetches and executes instructions; it contains a number of internal registers to enable this task to be carried out. The **Arithmetic and Logic Unit** —the **ALU**— is used to perform mathematical and bitwise operations upon data words, and produce condition flags describing the result.

Manipulation of the ALU and registers within the execution unit can not, of course, be performed by explicit instructions. Instead they are made to appear as locations within the computer's memory, allowing data and control information to be moved to, from and between them.

Memory not occupied by these units contains words of random access memory, for program and data storage.

Some locations in memory are declared as write only. Any attempt to read from these addresses constitutes an error, and the computer enters a halted

state. This detects some erroneous instructions, and is also used to explicitly halt the execution of a program.

There have been claims that the provision of active elements within the memory is 'cheating', and that there is really more than one instruction. The obvious counter to this is the fact that most computers have external devices attached to their memory busses, but no-one includes the capabilities of disc or display controllers when measuring the complexity of the CPU instruction set.

There are separate address and data busses. Multiplexing the two busses would have reduced wiring, but prevented the address and data being transmitted simultaneously. Each memory mapped unit would then have needed latches to buffer either the address or the data. This would have negated any construction gains made by multiplexing the two busses, and reduced the speed significantly. The only real advantage of multiplexing the two is that it reduces the number of pins leading out from a single chip CPU, which is not a problem in my implementation.

Figure 2.1: Block Diagram of The Ultimate RISC

2.3 Bus Widths

A number of factors were influential in the choice of widths for the two busses. All the current high performance microprocessors have thirty-two bit wide data busses, which seems to be current limit for production and packaging technology. It was known that the Pascal compiler would need registers which were 32 bits wide, and for efficient instruction execution the data bus had to be of a similar width. Such a wide bus needed more area and components than a sixteen bit bus, but was practicable due to the simplicity of the computer. The address bus size was derived from the data bus. For simplicity, the size of an instruction had to be a multiple of the data bus width. Two bits were needed to indicate the indexing modes. Using one 32 bit word per instruction, thirty bits had to contain two operands, so an address bus width of 15 bits was a natural consequence. It would have been possible to use two words per instruction, and have a 30 bit address bus, but for a simple prototype this was excessive. With each address indicating a 32 bit data word, then in fifteen bits 32768 locations can be addressed —128 Kilobytes of memory.

2.4 Interrupts

To ensure a fast response to events, most computers support an interrupt mechanism. This consists of one or more signals, which when asserted cause the computer to stop executing the current instruction sequence. The internal state of the CPU is stored and control is then passed to an interrupt handling routine. This program must service the request of the interrupting device before instructing the computer to continue normal execution. This feature is vital in any situation where external events require a rapid reaction. It can be also used as a means of job control by allowing a different process to execute after the

interrupt.

The provision of such a facility complicates the control unit significantly, for very little benefit in a prototype system. There is no justification for this feature given that the Ultimate RISC has no direct I/O capabilities.

Note that the Viper-1 microprocessor does not include interrupts either. This was a deliberate decision so as to guarantee that a program could not be corrupted by external sources. Thus my lack of interrupts can be viewed as a feature rather than an absence.

Chapter 3

Implementation Issues

Throughout the project I had always to bear in mind a number of issues:-

- The computer must be interfaced to an APM, and so built on a double size Eurocard.
- The budget for the project could not exceed £300
- The computer had to be designed, built and documented by late May.
- The choice of manual or automatic board construction.

These resulted in a computer which lacks some of the features which are normally expected.

3.1 Components

It was impossible to build my computer out of a few standard off-the-shelf VLSI components; it bore too little resemblance to existing systems. Fortunately, a large number of MSI components for building computers have long been available, and I made use of these. Individually quite small, together they occupy a large amount of area and have an excessive power consumption. It is worthwhile describing the main types of components here, rather than repeatedly describe them later.

3.1.1 Transistor-Transistor Logic (TTL)

TTL integrated circuits have been the standard building blocks for digital circuits for over a decade. They have a number of advantages:-

- widely available
- individually inexpensive
- simple enough to be flexible in application
- well documented
- operate at high speed

Many of these components need to be wired together to produce a useful circuit. Some types of TTL 'families' are low-power versions, being slow and suffering from fanout problems. I decided to use 74F (FAST) series logic, which were high power and high speed.

3.1.2 Programmable Array Logic (PALS)

These are integrated circuits which can be programmed with simple boolean logic equations expressed in the sum of products form [**?**]. They were used in situations where a standard component was not available. The fact that they were explicitly programmed made them correspond well to the formal specification, the equations actually being edited from the specification to the PAL programming format. This was important as these devices could not be reprogrammed —first time correctness had to be guaranteed.

3.1.3 Erasable Programmable Logic Devices (EPLDS)

These are a development of PALS, being erasable in ultra-violet light. This made them useful in the parts of the computer which needed reprogramming during development. They were much more expensive, and did not operate as fast as PALS do. The extra capital cost can be recuperated by re-use in later applications.

3.1.4 Serial Shadow Registers (SSRS)

These eight bit registers, manufactured by AMD, formed a central part of my design [**?**, **?**]. Designed for use in pipelined computers, they behaved as eight bit tristate registers. They also had extra circuitry designed to aid debugging. Each register had a shadow register which was invisible to normal operation. Each device could be instructed to copy the state of the outputs into this shadow, or overwrite the main register with the contents of the shadow. This shadow register could be shifted out one bit at a time, while new bits were shifted in. A number of these registers were connected together via these serial links to form a

chain. By connecting the register chain to the host computer, this computer was able to read and write the shadow registers. This permitted the host to discover the state of the Ultimate RISC, and to indirectly access its memory. In some respects the use of these SSRS produced a design which was a bit less efficient than would otherwise be possible, but this is made up for by the observability of the system. For example, the ALU could have been implemented in a third of the number of ICs, but would then have been observable only indirectly.

3.2 Wiring Methods

I had originally planned to have the computer automatically wired using the Department's BEPI solderwrap machine. This takes a circuit board with capacitors, IC sockets, and a list of the wiring between the pins, then wires up all the interconnections. The choice of a 32 bit data bus was made on the assumption that I would be using this system and so the extra wiring would not cause any construction delays.

As time passed I became aware of the disadvantages to prototyping on the BEPI machine. The main problem with solderwrapping was that it was impossible to modify a board once built, and extremely difficult to debug if faulty. It may have been faster to build than wrapping by hand, but there was the effort to be spent actually generating the files needed by the BEPI machine. What is more, there were even some doubts as to the correctness of the wiring which the solderwrap machine produced.

Taking these factors into account I eventually decided to wire up my computer manually. This allowed incremental development of the computer, which meant I was always able to demonstrate some part of the computer working, rather

than just a finely wired but useless board.

Chapter 4

The Host Interface

The Ultimate RISC has to be connected to a host computer to obtain data, to receive control signals and to return results. This host is able to examine and modify memory locations and the internal registers.

The obvious choice for a host computer was the departmental Advanced Personal Machine (APM) workstation. These were widely available for use and designed to permit extra boards to be easily installed. The compiler was also being developed upon this system, so the two machines could form a standalone system.

The traditional CS4 project method of connecting new boards to the APM is to use the standard APM backplane to provide access to shared memory. The disadvantage with this is that the APM bus is very difficult to interface to. Not all projects manage it successfully, even when using a known CPU or display controller. To try and interface a computer which could not be guaranteed to work reliably would have been futile. The method I used allows the host

computer to examine and modify both memory and internal registers on the Ultimate RISC computer. By making internal registers out of Serial Shadow Registers, the host can easily access them. Control signals are used to copy all registers to their shadows, to shift data and to selectively reload registers. The host also controls the clock signals of the computer, aiding both hardware and software development.

Memory cannot be read or written directly. Instead the host initialises the **address** register with the desired address —along with the **data** register during a write operation. The control unit is then instructed to perform the appropriate access. After a read operation the **data** register should contain the contents of the memory location selected.

Six bits of state information are sent in parallel from the Ultimate RISC to the APM. Five of these bits give the current state of the control unit. The sixth bit is sent from the memory section to indicate whether the current memory access is valid or not. The Ultimate RISC can thus be observed while operating asynchronously with respect to the host.

Three signals are sent to the Ultimate RISC's Control Unit by the APM. These instruct it to start and stop executing instructions, or to perform a memory access for the host.

The complete list of connections is given in table **??**, along with their positions on the Eurocard edge connector.

In all, sixteen control lines are needed —seven inputs to the host and nine outputs. This allows the Ultimate RISC to be connected to any host with a sixteen bit parallel I/O port.

The Real Time Systems M6809 co-processor board does have one of these de-

direction	signal	pin no.	description
Output	State 0	C2	these give the current state of the control unit
	State 1	C3	as a five bit number
	State 2	C4	
	State 3	C5	
	State 4	C6	
	$\overline{halt}$	C7	indicates the current read should cause a Halt
	SDI	C8	SSR chain input to the host
Input	SDO	C7	SSR chain output from the host
	$\overline{freerun}$	C12	instructs the computer to use its own clock
	clock	C13	the clock signal when not running freely
	$\overline{go}$	C14	execute an instruction or a memory access
	$\overline{load}$	C15	load **address** and **data** registers
	l/$\overline{s}$	C16	indicates the type of the memory access
	mode	C17	SSR mode selection
	dclk	C18	SSR shadow register clock
	$\overline{reset}$	C19	reset program counter

Table 4.1: Host Interface Specification

vices, and it is to this that the board is connected. It can in fact be connected to many simple computers, provided the software support is available.

4.1 The Monitor

The simplicity of the host interface is such that a sophisticated monitor program is needed to make any use of the Ultimate RISC. The development of this program went hand in hand with the building of the computer itself. When a new feature was built into the hardware, it was first tested with the existing

monitor features, and, once the protocol was established, supported by higher level routines.

The facilities offered include:-

- downloading and execution of programs on the 6809 board.
- manual manipulation of the interface lines.
- reading, modifying and copying back the registers of the Ultimate RISC.
- single stepping the computer's clock.
- reading and writing memory locations —including registers
- memory testing
- downloading code into the Ultimate RISC's memory
- instruction execution control
- emulated I/O on behalf of the Ultimate RISC.

The monitor is divided into two programs, each executing concurrently on separate processors and communicating via shared memory.

4.1.1 A Virtual PIA

The Ultimate RISC's host interface is connected to the Peripheral Interface Adaptor of the APM 6809 Processor Board. While most of the memory is shared with the 68000 processor this is not the case for the I/O devices. A short 6809 Assembly Language program is used to provide a virtual PIA for the 68000 microprocessor to use. This copies the input port of the PIA to a shared

Address	Description
0	the PIA register (0–3) to write to
1	the data to be written
2	the inputs to PIA port A
4	loop count to drive Port B; cleared on completion of loop
8	port B loop value #1
9	port B loop value #2
1E	the number of shared registers
1F	copy control =$80 to get the registers, $81 to write back
20	the start of the shared SSR image

Table 4.2: 6809 Shared Memory Locations

memory location, and uses data in other memory locations to update the PIA registers. This PIA contains two eight bit I/O registers. Port A of the PIA contains all the outputs from the Ultimate RISC, and the SDO signal. Port B is dedicated to the output of the eight control signals. This port is buffered to increase its drive capability. The buffer used on most of the 6809 boards is an LS TTL device, but fanout effects forced me to upgrade the buffer on the board I was using to one of higher power.

The first few memory locations of the 6809 processor's address space are dedicated to this inter-processor communication (table **??**).

When writing to a port of the PIA, the 68000 program writes the data first, followed by the register number plus 128; this high bit indicates a valid write request. When the contents of address 1 are cleared, the request has been processed and the copy of port A updated. The 68000 can use this to poll for the completion of an operation or test the operation of the 6809 board. This method of synchronisation in not particularly fast, so when controlling the clock

from the host, the maximum clock speed is only 1 KHz.

To increase speed the 6809 program contains some extra routines. These perform iterative operations without the need for constant synchronisation between the Virtual PIA and the main processor. The host's image of the Ultimate RISC's registers are stored in shared memory. The PIA program can be instructed to shift in a new copy, or shift out an updated copy. It can also be instructed to send an alternating pattern of signals out through port B a set number of times. This can be used to provide a faster clock signal of 50 KHz.

4.1.2 The Main Monitor

This is an extension of the CS2 6809 monitor, from which the Command Line Interface and the 6809 control operations are all taken. It has been extended to form a 2000 line IMP program with many more commands. There is also a status display at the top of the screen. This shows the state of all the inputs and outputs on the host interface and the state of the control unit as a mnemonic name.

To provide flexibility in development the data about registers and states are kept in separate files. This allows different configurations of both the control unit and of the registers to be used with minimal effort.

Object code can be downloaded into the Ultimate RISC's memory by the monitor. The format was designed to support both compiled and hand coded programs and is described in table **??**. It is when downloading code that the speed of the host interface becomes apparent; the liberal insertion of comments provides needed feedback as to the progress of the operation.

The monitor program also provides a rudimentary output facility for the Ul-

1. the file is stored as a text file, with normal restrictions on naming
2. all numbers are given in hexadecimal, one to a line
3. any line beginning with '**!**' is a comment, to be printed during downloading
4. @(address) states the shared printing address (default=$3FFF)
5. code sequences are represented as:-
   address
   code length
   (code)*
6. there is no limit upon the number of code sequences in a single file

Table 4.3: Code File Format

timate RISC. A downloaded program can specify a shared printing address. Whenever this program halts, the instruction which caused the halt is examined. If it is the instruction '**MOVE 0000,0001**' then this is interpreted as a request for output. The contents of the shared printing address are then printed on the terminal screen as an ASCII string, and the program is restarted.

The monitor is somewhat isolated from the Ultimate RISC, as the virtual PIA and the sixteen bit I/O port form a bottleneck in communication. The monitor's registers are only copies of the actual registers, and have to be updated explicitly, while the status line is of more use to hardware development than software. However, the monitor does provide adequate access to the Ultimate RISC.

Chapter 5

The Execution Unit

5.1 Design

Within this unit instructions are fetched and executed, under the supervision of the control unit. It is depicted in figure **??**. It contains a number of registers in order to carry out this task. A 15 bit **Program Counter** (**PC**) is used to locate the next instruction. This is incremented after each instruction has been fetched. To permit program branching, this register is a writeable memory location.

There are two index registers —the source index **X** and the destination index **Y**. Both are 15 bits wide and memory mapped. A fast adder is used to add the contents of the registers to the operand addresses.

Conditional branching is facilitated by a **Skip** register. This appears as a single bit memory-mapped register. When written to with the least significant bit

of the data set, this causes the next instruction to be skipped. If the bit is clear the following instruction is executed as normal. Without such a register conditional branching could still be performed by placing a conditional offset into the source index register, and moving the contents of the resulting address to the PC. This would be more cumbersome for a simple branch, but effective in multiway branches.

Internal registers are used during the execution of an instruction. These are not accessible by programs, but can be read and possibly altered by the host. They are :-

- **data** : 32 bits for storage of data while being moved
- **address** : 15 bits for buffering of the current location being accessed.
- **instruction** : 32 bits for storage of the current instruction. It is subdivided into the **source** and **destination**, each of which contains an **index** flag and a 15 bit **operand**

Their use is shown at the register transfer level in figure **??**.

5.2 Implementation

5.2.1 Registers

All registers except the PC are constructed out of Serial Shadow Registers. The **Program Counter** is recorded in a bank of parallel loading counters, which can be incremented by a control signal and reloaded by a memory write. To allow the host the ability to read the **PC** it is passed through a Shadow

Figure 5.1: Execution Unit

1. **address** ← **PC**

2. **instruction** ← (**address**);
   PC← **PC**+1

3. if **instruction.source.index**=1 then
   address← **instruction.source.operand** + **X**
   else
   address← **instruction.source.operand**

4. **data**←(**address**);
   if **instruction.destination.index**=1 then
   address← **instruction.destination.operand** + **Y**
   else
   address← **instruction.destination.operand**

5. (**address**)← **data**

Figure 5.2: Instruction Execution Sequence

Serial Register during the instruction fetch sequence —the **Program Counter Register** (**PCR**).

No Execution Unit registers can be examined with a memory read operation. This is inconvenient, as a program cannot determine the contents of the **PC**, **X** or **Y** registers. This prevents relocatable code being used, and complicates other operations. Supporting the reading of these registers would have used an extra eight tristate buffers, for which there was neither room nor money.

5.2.2 Connections

The outputs of the **PCR** and the instruction operands are all connected to the inputs of the **Address** register. An adder made from four 74381 ALU units and a carry lookahead generator performs the adding of offsets to indexed instructions. The **Source** and **Destination** operands and the **X** and **Y** index registers are multiplexed into this adder using the tristate outputs of the registers. The outputs of the adder are fed to a tristate buffer. Another buffer exists to bypass the adder completely, for non-indexed operands. The selection of whether to index the offset or not is controlled by the index flag of each operand, without the control unit's intervention.

5.2.3 Skipping

Instruction skipping is performed independently of the control unit. The input to the count signal of the **PC** is multiplexed between a count signal from the control section and bit zero of the data bus. Signal selection is controlled by the address decoder. On recognising a write to the **Skip** register this decoder connects the databus to the count input for one clock period only. The program

counter is then incremented only if the least significant bit if the data word is set.

Chapter 6

The Control Unit

6.1 Design

For the computer to work correctly a number of signals need to be sent to different components at different times. Most of these signals are generated by the control unit. It implements the instruction fetch and execute sequence, and performs memory accesses for the host. It is a finite state machine which uses four input signals to select the next state, and produces fourteen outputs (table **??**).

The design of this unit had to wait until the rest of the computer had been designed on a component by component basis. I had originally envisaged that the Control Unit would need to know whether the current operand was indexed or not, or if it should be skipped altogether. By hardwiring these features into the Execution Unit, the Control Unit operates without knowledge of the current instruction.

It is supplied with the $\overline{halt}$ signal from the Memory Unit, to indicate the validity of the current read access. If reading a memory location is not possible this signal is pulled low, and the Control Unit abandons the current access and enter a halted state.

6.2 Implementation

The Control Unit is implemented as a single 24 pin EPLD. This can be in five operating states:

- stopped
- instruction execution
- loading the Data, Address and Instruction registers from their shadows
- performing a read operation for the host
- performing a write operation for the host

The read and write operations, and hence also instruction execution, take more than one clock cycle to be completed. The actual number of cycles depends upon the speed of the clock relative to the memory access time. The Moore Machine within the EPLD is therefore designed to contain up to 32 states —extra wait states need to be inserted as the clock speed is increased.

6.2.1 Clock

The Control Unit, address decoder and Program Counter are all supplied with a common clock. The host inputs are synchronised against this clock to ensure

direction	signal	description
Input	$\overline{go}$	from the host - execution control
	$\overline{load}$	load **address** and **data** registers
	l/$\overline{s}$	indicates the type of the memory access
	$\overline{halt}$	indicates the current read is invalid
Output	State 0	these give the current state of the control unit
	State 1	and are all zero on power up
	State 2	or after a halt
	State 3	
	State 4	
	r/$\overline{w}$	memory access direction
	ms	requests a memory access
	loadD	load **data** register
	loadM	load **address** register
	loadI	load **instruction** register
	loadPCR	load **PCR** register
	incPC	increment the Program Counter
	$\overline{s}$/d	select source or destination operand
	$\overline{OeOP}$	output enable current operand

Table 6.1: Control Unit Inputs and Outputs

that no signals change so close to its rising edge that setup times are violated.

A system clock is generated on the board using an oscillator module. The output of this clock is then fed into a multiplexer. The multiplexer selects, under host control, to use either the on board clock or a host generated clock signal.

6.2.2 Difficulties with EPLDS

There was major confusion over which EPLDS can be programmed within the department, to my eventual detriment. It was never clear whether the software to program the devices which the department has —the A+ package— would actually be able to program the fast EP610 devices, or just the slower EP600 ICs. Claims that an EP610 was programmed by a previous year's student were supported by a salesman at the suppliers who said it was possible. There were, however, some rumors that this was not in fact possible with our old programmer and software. Eventually, after I had sent off an order for two EP610 EPLDS, it was confirmed that the programmer would not be able to program the faster devices. Fortunately, I was able to change my order to EP600 EPLDS. These have a maximum cycle time of 45 nS and a 38 nS setup time, so are neither fast nor responsive. This limits the performance of the computer significantly, reducing its maximum possible clock frequency from 40 MHz to 20 MHz.

Chapter 7

The ALU

7.1 Design

For a computer to be capable of effective operation, it needs the ability to perform actual processing of data. It has long been known that the ability to compare two items and act upon the result is sufficient for effective computation —Turing Machines are based around this concept. It would therefore have been possible to build a basic comparison unit, and rely on software to derive mathematical and logical operations. This would have been unreasonably inefficient. All realistic computers have hardware dedicated to evaluation of these functions. These Arithmetic and Logic Units (**ALU**) normally perform at least integer addition, subtraction and the standard boolean functions of two variables. More powerful units are capable of high speed multiplication, or even manipulate floating point numbers.

At the start of the project I was offered the possibility of using a single chip

64-bit floating point ALU from AMD (AM29C327) [**?**, **?**]. This would have produced impressive performance figures, but I decided that it would have been unworkable, since it was designed for a triple data bus and needed 31 bits of control information every cycle. A single bus system would have been unable to use this device effectively.

Instead I designed a very simple ALU, since this made formal specification possible. The unit was built from eight bit sliced TTL ICs, each of which operates on four bits. When connected together via a two level carry lookahead generator, they perform operations on 32-bit words.

This is sufficient for many purposes, except that the ability to shift a word right was needed in iterative multiplication and division algorithms.

The result of the ALU had to be stored until re-used in later instructions. The state of this result, whether zero or negative needed to obtained in a form which could be passed to the Skip register. Arithmetic overflow and carry flags were also desirable, detecting results too large to be represented in 32 bits.

7.2 Implementation

The design of the ALU is shown in figure **??**.

An Accumulator stores the output of the ALU between operations. This can be read as a memory location. The contents of the Accumulator are also used as one of the inputs to the ALU, so only one other argument needs to be supplied per operation. This accumulator is built out of four SSRS, so can be read directly by the host.

A number of bit sliced ALUs were available with built in accumulator registers. For example, the AMD AM2901 ([**?**]) or the TTL 74F681 ALU bit slices, would have provided enhanced performance with less components and wiring. Using these would have prevented the host examining the Accumulator directly. Instead I used 74F381 ALU/function generators in my design. These only perform basic operations —addition, subtraction, and, or, exclusive or, preset and clear. Three control signals are used to select a function.

Between the outputs of the ALU ICs and the Accumulator is a bank of five PALS. Normally these pass the result straight through, each PAL checking if the bits passed though it are all zero or not. They can also be instructed to shift the result —including the carry flag— one bit to the right; this shifting is controlled by a one bit signal. This post shifting allows a normal operation to be combined with a shift, to make unusual functions such as 'subtract and divide by two'.

The results of the five zero tests along with other signals are fed to another PAL, which produces values for a Condition Code register (**CC**), constructed from a Shadow Serial Register. The PAL generates a zero flag when all five slices of the result are zero.

Overflow

An arithmetic overflow is where a signed number's sign changes due to too large an addition, subtraction or shift.

My design of an ALU does not detect signed overflow, despite the original intent to do so. I had originally acquired equations from my CS3 notes to detect overflows using a PAL. While specifying the system I realised these equations

Figure 7.1: The ALU

only detected overflow on signed addition. To detect overflow in a multi-function ALU, one must compare the carry between bit 30 and bit 31 of the result with the most significant bit, an overflow occuring if the two differ. This can not be done with the 74F381 bit-sliced devices, as this carry is internal. I have discovered that AMD make a special most-significant-slice version of this bit-sliced ALU which does detect overflows internally. The result of this check would however become confused if shifting was performed after the operation, so would not always be reliable.

Note that even if the ALU did produce an overflow flag, the software would still have to check it after every operation. A number of implementations of languages do not do this because of the overhead this entails; APM Pascal and Standard ML are two such implementations.

Memory Interface

Seventeen addresses are allocated to the ALU, as shown in table **??**. One of these addresses returns the current value of the Accumulator whenever it is read. The remaining sixteen addresses all apply a different function between the accumulator and the word moved to the selected address. This is accomplished by wiring address bus lines directly to the ALU and the PALS.

It is not be possible to directly load the accumulator, but a two instruction sequence clears it and then adds a number to the now empty accumulator.

Condition code flag manipulation is supported: reading any of the sixteen function addresses returns one of the condition code flags in the least significant bit. These results can be passed directly to the Skip register for conditional branching. Before performing a subtraction the carry flag has to be set to true,

while for other operations the flag has to be cleared. An address is provided to enable this; when it is written to, the least significant bit is passed to the carry flag.

Chapter 8

Memory

8.1 Overview

In most computers memory is used for storage of programs and data. In many microcomputers peripheral elements are also interfaced to the memory busses, so that reading or writing certain locations actually controls the peripherals. This obviates the need for special I/O instructions and control lines. The technique is called *memory mapping*. It is useful where the composition of systems varies widely, as different peripherals can be easily memory mapped. Example peripherals include a display controller, a disc controller, Ethernet communications link, keyboard decoder...etc.

Since the only instruction available upon the Ultimate RISC is a memory to memory move, *all* functional elements of the system must be memory mapped, including the computer's main registers (table **??**). While this makes the control unit simple, part of the burden of complexity is passed to memory, remaining

to extract a price on performance.

8.2 Address Decoding

The memory of the computer needs to be informed when a memory access is required. It must be told the direction of the access, the address to be used, and, on a write operation, the data which it must write.

During a read cycle it will return either the data stored at that location, or a signal indicating the address was invalid.

Some write operations trigger one or more signals to different units of the computer, to update internal registers.

A means is needed of interpreting the address and control signals to generate the required outputs. This is called *address decoding*.

The first decision to be made when designing the address decoder was the access protocol. A *synchronous* memory relies upon the access being completed a certain time after it begins, whereas an *asynchronous* memory requires the computer to wait for an acknowledgement signal. I decided to implement synchronous memory, which was simpler but did rely upon the speed of memory being known. The fifteen address bits and a $\mathbf{r}/\overline{w}$ signal are used to indicate the location and direction of the access. A further signal, **memory select** (**ms**) is used to actually request a memory access.

To write to an address the computer loads the **address** register with the address and enables the output on the **data** register. When both are known to be valid **ms** is asserted with $\mathbf{r}/\overline{w}$ low, and decode begins. The signals must remain until

Unit	Address	Name	Function
Execution Unit			
Write Only			
	0	PC	program counter
	1	SKIP	skip register
	2	X	X index register
	3	Y	Y index register
	4–7		duplicates of the above
ALU:			
Writing			
	8–F	Carry	$Carry \leftarrow data_0$
	10	CLR	$Acc \leftarrow 0$
	11	SUBR	$Acc \leftarrow data - Acc$
	12	SUB	$Acc \leftarrow Acc - data$
	13	ADD	$Acc \leftarrow Acc + data$
	14	XOR	$Acc \leftarrow Acc \oplus data$
	15	OR	$Acc \leftarrow Acc \vee data$
	16	AND	$Acc \leftarrow Acc \wedge data$
	17	SET	$Acc \leftarrow$ $FFFFFFFF
	18–31		10–17 followed by a one bit rotate right
ALU:			
Reading			
	8–F	Acc	Accumulator
	10	Z	Zero Flag
	11	N	Negative Flag
	12	V	Overflow Flag (not supported)
	13	C	Carry Flag
	14–31		10–13 repeated
Memory:			
bidirectional			
	2000	RAM	56K Random Access Memory
	3FFF		

Table 8.1: Memory Map

it can be guaranteed that the write will be completed. This means a write operation must include a delay to allow for ALU operations.

To read an address the protocol is similar except the $\mathbf{r}/\overline{w}$ line is set high and the **data** register is not output onto the bus; at the end of the read either the **data** or **instruction** register is loaded with the value on the data bus.

8.2.1 Address Decode Implementation

Most addresses are decoded using EPLDS, rather than PALS. This is slower but provides for later expansion of memory. One EPLD is used for decoding the simple signals which can be asserted throughout the cycle —the loading of Execution Unit registers, and the reading of the Accumulator and condition flags.

A further EPLD is used for the generation of the one-off signals which load the **ACC** and **CC** registers after a function evaluation. It also controls the source of the count signal of the **PC**. This EPLD is clocked to enable it to keep track of time during a write operation, and contains a finite state machine.

The time to access the Random Access Memory is longer than for other units, so in order to eliminate unacceptable decoding delays, the $\overline{write}$ and $\overline{Oe}$ signals for this are generated using hard wired logic. This has a propagation delay of under ten nano-seconds.

To ensure there are no glitches during the decode, I found that the direction of the access must be changed prior to the assertion of **ms**. Problems were caused when the control unit EPLD generated some outputs before others.

Figure 8.1: Memory Access Protocols

8.3 Random Access Memory

The area of Random Access Memory needed to be fast and easy to use. This is why I decided from the outset to use Static RAM rather than Dynamic RAM, which while cheaper and denser was slower and much harder to interface to. Deciding how much Static RAM to use, and at what speed was a major source of problems, due to cost and availability considerations. I had originally envisaged that the computer would have a full 128kB memory from four 32k*8 Static RAM ICs. To increase the projected speed of the computer I decided to use very fast memories, but being more expensive I had to have a smaller amount. The suppliers R&R advertised 16k*4 SRAMS with 25 nS access time, eight of which would have been ideal, especially with a cost of only £5.50 each. After much telephoning it became clear they did not have any in stock. Another supplier did claim to have these memories, but £15 each was too expensive.

Rather than rely on promises of availability I based my design on 8k*8 memories of 45 nS. Four of these provide 8192 words, which places a tight limit upon the size of code which can be executed.

Chapter 9

The Formal Specification

9.1 Methodology

The computer has been formally specified at the architectural level. The specification methodology used was **Lambda** [**?**]. Designed especially for hardware specification, it is implemented as an ML based system upon the Sun workstations. A semi-automated theorem prover forms the core of Lambda, which enables properties of specifications to be examined, and reifications to be verified.

The system is designed to synthesise a working design from a behavioural specification, by reifying the design until it describes individual components —*forward synthesis*. In such a way the correctness of a design can be guaranteed without performing a verification. Written in the language ML it has a similar syntax, but with extensions to the language. This enables part of a specification to be executed as ML functions, allowing the hardware's properties to be simulated

in software.

The Lambda specification language is more powerful than an executable programming language. Rather than describing a function or procedure to convert from the input to the output, one just specifies preconditions and postconditions.

For example, the function:-

```
fun square_root (x:Natural) = iota y. y*y==x;
```

describes the square root function in Lambda but not ML. Timing constraints can also be included into *rewrite rules*

```
val sqrt_unit#(x,y)=
        forall t.
             y (t+100) == square_root (x t);
```

This describes a combinatorial square root unit which outputs at time t+100 the square root of the input at time t.

Functional units can be joined together by use of common variable bindings

```
val double_sqrt_unit#(x,z)=
      sqrt_unit(x,y) /\ sqrt_unit(y,z);
```

Such functions, along with the definition of the types of the variables of X,Y and Z, can be parsed by Lambda to produce an environment of rules. These rules can be used to prove hypotheses, such as that the time for a double_sqrt_unit to evaluate an expression would take 200 units of time.

This system allows someone to specify any component or module as a collection of related inputs and outputs. A number of components can be linked together to form a larger module, or a complete design.

Unfortunately, none of the components I used had been formally specified. While I produced some specifications based on the informal specifications in the databooks their correctness can never be guaranteed.

Specification from scratch is an extremely slow process, and I was not able to describe the whole computer in this depth in the time available. I described the operation of the computer at a very high level, and then expanded the description of the ALU to a greater depth.

9.2 First Specification

My first specification was based upon an example specification of a simple computer in the Lambda Manual. It was written in a early version of Lambda, which had a syntax more complex than that of ML. First it was necessary to specify the types of data which the computer dealt with. Abstract datatypes of 15 and 32 bit integers were defined without giving their internal structure. Allowable operations —comparison and addition for 15 bit numbers, all ALU operations for 32 bit numbers— were stated as existing and being total. Their exact functions were not given. Random Access Memory was then defined as a function of 15 bit addresses to 32 bit data words.

The computer can at any moment in time be described by the contents of every register and memory location. As an instruction is executed this state changes, unless it is halted or in an infinite loop.

The state of the computer was described as a tuple of

```
<memory, execution unit state, alu state>
```

where the execution unit state was

```
<PC,X,Y,skip,halt>
```

and the ALU state was

```
<ACC,z,n,v,c>
```

The operation of the whole machine was given as a transition from state to state. Each transition was caused by the execution of a single instruction. The most complex part of this specification was the description of the read and write operations. This was because of the memory mapping of registers. The read function was supplied with the computer state and a 15 bit address to return a 32 bit integer. A separate function was used to validate the address prior to the read access. The write function took as parameters a state, an address and a new value, returning a new state. This allowed both registers and RAM to be updated.

The transition from one state to another during instruction execution was described by a function which fetched the next instruction, and incremented the PC. It then calculated the source address and moved its contents to the destination address. If the halt flag was set this function did nothing. Before each read the address was validated —any illegal access terminated the function and set the halt flag. If the skip flag was set the program counter was merely incremented and the flag cleared; no instruction was executed.

This does actually resemble the actual process of instruction fetch and execute, except that the halt and skip flags do not actually exist. Skipping is performed by hard-wired logic, and the halt state is merely another state within the control unit's Moore Machine. These differences are invisible to the user. At this level the operation of the computer is being described, even if it differs slightly from the actual implementation.

9.3 Second Specification

By Christmas a new version of Lambda was available, with a syntax more similar to ML. The design of the Ultimate RISC had become clearer, partly through the initial specification, but also as I designed the ALU.

I therefore upgraded the specification to support both the new notation and to be consistent with the revised design.

This was done by first expanding the 15-bit and 32-bit integer abstract data types to boolean tuples, with functions for conversion between these representations and that of natural numbers.

I then wrote all the operations performed by a 74381 ALU IC as functions acting upon boolean four-tuples. A general function was written to apply the operation selected by the control lines. The production of carry signals from the 74182 carry lookahead generators were also specified. In both cases the specifications were based upon data sheets from TI and AMD. It was then possible to describe the ALU components by relating outputs as the result of the functions applied upon the inputs of a previous time —the temporal difference being the propagation delay of the device.

The logic equations of the PALS were all specified likewise, enabling the entire combinatorial portion of the ALU to be accurately described.

The remainder of the specification was derived from the first specification of the Ultimate RISC.

I did not go about proving the two specifications were identical. Nor did I try to prove properties of the new specification, such as the non-folding of RAM addresses and the persistence of data within. Given estimates of the time to verify the correctness of specifications of other computers, there would have been no possibility of both verifying the specifications and attempting to build anything. The verification would probably form a complete project, requiring someone far more experienced in machine assisted proofs than myself. Instead I produced a simulation, by modifying the specification to execute in ML.

To enable my specification to be executed I had to remove all instances of postconditions and timing constraints. The other problem was 'feedback'. The 74381 units produce signals which are passed to the carry generators. These then return a carry signals to be evaluated with the earlier inputs. These were simulated by iterations of the functions.

I also wrote a 'monitor' for the simplified specification which provided a front end for the simulation with facilities such as memory read/write, instruction assembly and dissassembly, register manipulation and program execution.

There were subtle differences between ML and Lambda. Notably ML's integers were more restricted than Lambda's type Natural. Both the Edinburgh SML and the faster New Jersey ML had only 32 bit signed integers, rendering conversion between these numbers and 32 bit boolean tuples difficult. The specification only executed satisfactorily in PolyML, which was prone to unannounced field upgrades, so that functions available in February did not work in April.

9.4 Summary

Overall the specification and simulation comprise 1400 lines, and are somewhat more difficult to understand at a glance than P-CAD circuit diagrams. Much of the code is devoted to mathematical and component definitions. For the success of hardware specification languages I believe it will be necessary to produce libraries of verified maths functions, components and standard cells.

It is not a complete specification of the hardware of the Ultimate RISC. For this the computer must be described as a collection of units, communicating via control signals and synchronised by a clock. The time delays of all operations must be specified, along with the ability of the host to control the clock and write to registers.

The simulation has been used to test the operation of the ALU, especially the programming of the PALS, uncovering a couple of mistakes which could have proved costly. It also demonstrated that the carry flag had to be set prior to a subtraction, a fact the compiler writer needed.

While I have not performed any proofs, specifying the computer was a valuable exercise. Describing the machine at a high level, I was forced to consider many details which any other method of description would ignore; this clarified the hardware implementation.

The specification of an existing design did seem easier than the forward synthesis method for which Lambda aims to provide. Forward synthesis should reduce the number of proofs required, and thus increase the speed of designing with formal methods.

Even without the verification between levels, a specification which describes the

computer is of use describing the system to a software developer. For this to be the case the software designers need to be able to understand the notation. This is an argument in favour of using a more widely known notation such as **Z**, which seems to be primarily for software development. If programs were specified in the same notation as the hardware the two would be able to be integrated in order to prove facts about the combined system.

Ideally the specification should have been continued till every component was specified along with the interconnections. A netlist could have been extracted and sent to the BEPI machine for automated wiring, and the PAL and EPLD programs also generated. If these processes could be at least partially automated, then the Lambda system could form the core of an an automated design and manufacture system. Without such a system, manual intervention —whether P-Cad design or wire-wrapping— introduces elements of risk. For the full benefit of formal methods the implementation must match the specification.

Chapter 10

Performance

To actually judge the effectiveness of a particular architecture, one must evaluate it in a form with which it can be compared against other computers. A number of measures exist to do this, each with their own particular qualities. For integer performance a common measure is instruction throughput, measured in *Millions of Instructions per Second*, (MIPS). It also has a commercial derivative *dollars per MIP*. There are standard benchmark suites, such as *dhrystones*, which execute known programs a large number of times. These measure the compiler-computer combination. While the figures benchmarks produce are useful in advertising material, single numbers should not be used as the sole evaluation criteria. Of equal importance are such factors as I/O access rates, mean time between failures and the quantity of available software.

My simple prototype provides little other than instruction execution, so only measures of instruction throughput are possible.

The speed of the computer is limited by the speed of the components within.

All delay estimations were based upon the data sheet, they have not actually been measured yet. Typical case timings were assumed for devices in series; this was justified by the Central Limit Theorem. Wherever a large number of devices were connected together in parallel, I had to assume that one of the devices would exhibit the worst case of behaviour, delaying the entire bank.

10.1 Execution Unit Delays

The most significant delay within this unit is caused by the the adding of an index to the source operand, since this can not be overlapped with any other operation. This enforces a delay between the loading of the instruction and the loading of the address register with the source of 45 nS. Other operations, such as calculating the address of the next instruction, or the destination, are carried out during memory accesses, and hence are non-critical.

The propagation delays of the address and data registers require that these must be loaded 20 nS before the **ms** signal is asserted.

10.2 ALU timing

The time taken by the ALU to evaluate a function depends upon the operation and the inputs. The fastest operations are **preset** and **clear**, which do not depend upon the data inputs at all. The slowest operations are arithmetic operations, since these must use the carry lookahead system. The slowest of these operations are those causing a carry to ripple along all 32 bits. Such operations could require up to 30 nS to be evaluated. To pass this result through the shift unit requires another 25 nS, at which point the Accumulator can be

loaded. The evaluation of the condition flags takes another 25 nS. With the set up time of the CC register, the ALU should take 90 nS from the presentation of valid data and functions to full evaluation of any operation. Currently two EPLDs are used in the ALU, one to shift part of the result, and one to evaluate condition flags. These cause an extra 20 nS delay, but can be easily programmed into PALS, so are not included in this performance estimation.

10.3 Memory Delays

10.3.1 Memory Read Accesses

The speed of the RAM used, combined with the hardware decode of its enable signals causes it to be as fast as the other memory mapped units. This is because the other units' control signals are decoded through a slow EPLD which counteracts their faster responses.

If the address and the read signal are valid at least 10 nS prior to the assertion of **ms** then a read access only takes a further 50 nS.

10.3.2 Write Access

The write time is longer that the read time, due to the necessity of asserting the address and data lines for the duration of an ALU operation. The ALU has a propagation delay of 90 nS; with the setup time of the address and data registers a write cycle must last at least 100 nS. This is the delay between the address and data being valid to the loading of the Accumulator and the CC register. The duration of the **ms** signal can be shorter than this —a minimum

Parameter	Symbol	delay(nS)
RAM read decode	t_{ms_ramoe}	8
RAM write decode	t_{ms_ramw}	12
RAM output enable	t_{ramoe_lz}	20
RAM output tristate	t_{ramoe_hz}	20
RAM write	t_{ramw_w}	25
EP310 decode	t_{ms_ep310}	35
EP600 decode	$t_{ms\_clk\_ep610}$	22
PC write	$t_{ep310\_clk\_ldpc}$	2
ACC output	t_{ep310_oeacc}	9
CC output	t_{ep310_oecc}	7

Table 10.1: Memory: AC Characteristics

of 50 nS, or two clock cycles. Buffering the address and data lines into the ALU would shorten this write time to the same as that for a read access.

10.4 Control Unit

Since the Control Unit must generate the signals for the memory accesses and the Execution Unit, it can only introduce more delays. A synchronous circuit, it is dependent upon the system clock. The delay between states, and thus signal generation, is always a multiple of this clock speed. When clocked at a frequency below 20 MHz, the control unit is the sole limit on performance. Above this frequency extra states have to be inserted into the state machine to wait for memory accesses. Above 30 MHz the addition of the source index would require an extra wait state. This could be eliminated by feeding the index signal in as an input to the Control Unit; a spare pin has been left for this purpose.

However, the operational speed of the EPLD used rendered this unnecessary.

10.5 Maximum Performance

Ignoring control unit delays, the minimum time to execute an instruction is easily calculated.

$$\begin{aligned} t_{ex} &= 2 * t_{read} + t_{write} + t_{index} \qquad (10.1) \\ &= 100 + 45 + 60 + 60 \\ &= 265\ nS \end{aligned}$$

(10.2)

This would give an instruction throughput of 3.8 MIPS. This can never be achieved due to the control unit delays. A 40 MHz EP610 EPLD could execute an instruction in twelve steps, 300 nS. This would give a throughput of 3.33 MIPS. Due to the lack of a programmer for such a device I was forced to use a slower EPLD with a maximum clock speed of 50 nS. This increases the time to execute an instruction to 400 nS, reducing the throughput to only 2.5 MIPS at most.

10.6 MIPS as a measure of performance

Comparing MIPS between processors is a rather dubious affair, especially with RISC computers.

Manufacturers of traditional CISC computers justify their apparent low performace by claiming that although a complex instruction set may have a lower

throughput, the power of each instruction means that fewer instructions are needed.

This may make it difficult to compare a computer with only a single instruction against such computers.

However, RISC architectures are designed support the most common instructions of CISC computers, so the number of instructions used should not be much greater. One can balance out this difference in code size by including the number of instructions in any measures of performance.

Comparisons have been made between the object code produced by the Ultimate RISC's compiler and M68000 object code produced by a compiler with the same front end. These indicate that that only twice as many clock cycles are needed to execute a program on the Ultimate RISC as on the more complex microprocessor. If this ratio holds for a large sample of programs then one can conclude that an Ultimate RISC computer should show comparable performance to a 68000 microprocessor operating at half the speed.

Chapter 11

Conclusions

One can draw a number of valuable conclusions from this project, regarding the architecture, my implementation of it and of the rôle of formal methods in hardware design.

11.1 My Implementation

My implementation of the Ultimate RISC provides a thirty-two bit wide processor and memory on a single APM board. The only use made of VLSI in the design was the memory chips.

Its method of communicating with the APM, while simple and effective, is extremely slow. The host can only perform a small number of memory accesses per second, making data transfer a time consuming task. This interface is independent of any particular host; with the appropriate software it can be

connected to many common microcomputers.

It lacks any genuine I/O facilities, although these can be emulated by the monitor program.

There is no means of addressing data of any size other than thirty-two bit words. Data has to be stored aligned with the long memory words, or packed and unpacked with difficulty. This makes storage of data items such as strings and bit-vectors highly inefficient, which is a pity given the small amount of memory available.

Of major inconvenience is the inability of instructions to read the **PC**, **X** and **Y** registers, or write directly to the **Accumulator**. This could be rectified by adding another fourteen tristate buffers - two each for the Execution Unit's registers, and eight to multiplex the **Accumulator** inputs.

These extra demands do not actually increase the functionality of the computer in any way. All the problems can be solved in software alone, using up a bit more memory. Hardware extensions would make the computer larger and more expensive.

Designing a computer is a matter of trading off the requirements for hardware cost and complexity with those for performance and ease of programming. Should extra components be added to make programming slightly easier? I was persuaded to include indexed instructions because of the immense difficulty of writing reliable self-modifying code. The requests for bi-directional access to registers by programs I declined because the cost outweighed the gains.

The Ultimate RISC is meant to be simple but powerful, and in some respects it meets these objectives.

I could not have specified, designed and built a more complex machine within the timespan of a single year. It would not have been possible for me to build a computer with more features and still have it fit on a single APM board, or cost within the budget. It would have probably been possible to build a more complex computer if I had halved the width of the data bus and all associated parts. This would have reduced the performance drastically for very little benefit. This fact is in itself a justification for the entire *less is more* philosophy of RISC architecture, which this project has taken to its very limit.

11.2 Further Development

There is little further development which can be performed on the single board. By reprogramming the control unit one could experiment with features such as delayed branching and skipping, which could save a single cycle from each instruction. Other experiments could be made with a bit of judicious rewiring.

11.2.1 Extended Indexing

Currently the most significant bit of any address is ignored, due to the small amount of memory available. This spare bit of every operand could be diverted to the selection of indices, to allow the X and Y registers to be added to either operand. This may well increase the flexibility of the indexing scheme. The reason I have not already done this is to maintain consistency between the computer and its specification.

Since a full function ALU is used to perform the adding, I could also experiment with different functions between the index registers and the operand addresses.

The boolean operations might be useful to implement some kind of protection mechanism to prevent access to certain memory areas. Some means of specifying the function to be applied is needed, and rather than use up valuable instruction space, I could add a register which would be loaded up with the function control signals. Of course, any of these operations could equally well be done using the existing ALU so there is no real justification for this modification other than idle curiosity.

11.2.2 A Floating Point Unit

While most commercial floating point coprocessors are tightly coupled to the appropriate scalar processor, Wietek are said to manufacture a floating point unit (Wietek WTL 1167) which can be memory mapped into a microprocessor's address space. It should be possible to interface such a device to the Ultimate RISC. The only drawback is that it would have to be placed onto a second board, which would increase communications delays and require much extra wiring. If the co-processor had an access time of longer than the current ALU then the cycle time of the entire memory would have to be increased by including wait states within the control unit. This may be offset by the ability to process floating point numbers at speed.

11.2.3 Extended Addressing

If expanding onto a second board, it would be useful to add extra memory, be it RAM or EPROM. It is in fact possible to increase the address space of the computer to sixteen bits merely by changing eight wires. The execution unit is capable of processing sixteen bit numbers, with the **X**, **Y** and **PC** registers being this length. The only 15 bit quantities are the operand addresses of

instructions. These will have to be added to one of the index registers to give a full size address. In this way the index registers provide a virtual address extension of a 32K Word area into a 64K Word real memory.

11.2.4 Software Support

There is no point in extending the hardware capabilities of a computer if it can not be well supported in hardware. The main software environment for the Ultimate RISC is not an assembler but a PASCAL compiler. This would have to extended to take full advantage of any rebuilding.

It would be worthwhile developing the monitor program, to provide more powerful I/O facilities, such as file handling. If programs on the Ultimate RISC could access data in files, then the small amount of memory available would be less of a limitation.

11.3 The MOVE architecture

11.3.1 The Advantages

The Single Instruction Computer is the simplest computer one can describe as having a von Neumann architecture. It has both a memory for programs and data, and a unit which fetches and executes instructions.

Making all instructions memory to memory is of use in applications which, rather than being computation intensive, need to move a large amount of data around quickly. These include image and text manipulation. I have also heard of claims that this architecture is of use in AI applications [**?**]. This is not as

far fetched as it seems when one considers the amount of time which functional programming languages such as LISP and ML spend performing garbage collection upon heaps. AI programs do not tend to exhibit the same *locality of reference* as traditional programs have, upon which many methods of increasing performance rely on.

It is an utterly flexible architecture. Since all functional units are memory mapped, one should be able to pick-and-mix the units to use for a particular application. A floating point unit could be added for mathematically intensive operations. Display, communications and other I/O connections could be added to produce a standalone system. One could even experiment with more unusual units, such as an array processing area of memory.

As more functionality is provided the rôle of the control and execution units becomes relegated to that of routing the results from one unit to the inputs of another. The rate of transmission becomes limited by the bandwidth of the single bus. One could improve the throughput by storing the program in a separate area —a Harvard Architecture—, but then it would not be possible to experiment with self-modifying code. This might be useful if one wished to use the computer in a Digital Signal Processing application, where operations were to be performed on a stream of data at high speed.

The point is that while communicating with external units normally causes performance degradation, this is not the case with this architecture. Access to any unit should take equal time. This may not be so much a feature as the major drawback of this architecture. The performance of a single instruction computer will tend to be less than processors containing a floating point ALU very tightly coupled with their integer processor. This seems to be a common feature of many recently announced microprocessors whether RISC or CISC e.g Intel i860 & i486, Motorola 68040 & 88000, MIPS R3000 & R3010.

The throughput of the execution unit could be increased by pipelining it. Using multi-port memory would enable two read accesses and a write access to take place concurrently, so three instructions could be overlapped. As well as increasing the hardware cost and complexity, this would produce problems such as delayed branches and skipping, and read after write conflicts.

11.3.2 The Disadvantages

Being such a simple implementation of the von Neumann architecture, it is completely at the mercy of its infamous drawback, *Memory Bandwidth*. The limiting factor of this design is the speed of memory accesses. Most computers, especially mainstream RISC architectures use register-register operations almost exclusively. Memory accesses are often performed through a cache, which can read or write a number of locations in a single burst. This tends to bias these processors to applications where more then one operation is to be performed on a single data item.

With every instruction in an Ultimate RISC requiring three memory operations, almost the entire bandwidth of memory is consumed by a single execution unit. This would makes it difficult to share the bus with other devices such as a DMA controller. Impressive performance figures can be achieved by using very high speed memories, but most systems use larger quantities of slower memories. While caching can be used to accelerate accesses to traditional inactive memories, any addressing of active memory elements would have to bypass this cache, and so reduce the gain.

11.3.3 VLSI Implementation

While this is a compact and effective design to build out of MSI logic, the future of high performance computing lies in VLSI. In these designs off-chip communications are to be avoided wherever possible, propagation delays being an order of magnitude bigger than for internal connections. In VLSI one benefits by placing as much functionality upon a single chip, rather than distributing it across a number of ICs.

Implementing an Ultimate RISC in VLSI would not be difficult at all, and take up much less area than any other 32-bit microprocessor would. One could then include on the same substrate an area of RAM and a number of functional units. Such a computer would still need access to external memory, slowing it down some of the time.

If the area of a single control and execution unit is small enough, one could fit a number of them onto a single chip. Extra functional units could be provided to support interprocessor communications —a region of shared memory or special channel addresses. While this may not offer more power than available multiprocessor designs, it could be used to make more efficient use of functional units by sharing them among processors. Consider: the ALU is only ever active during the write cycle of an instruction alone. It is therefore unused at least 2/3 of the time. If three execution units were provided with their own registers multiplexed into the combinatorial part of the ALU during their individual write cycles, then by operating the three units exactly one instruction phase apart, the ALU could potentially be kept 100% busy.

Another possible application of a VLSI version of the Ultimate RISC would be as part of a standard cell library, for then the full flexibility of the architecture becomes apparent. A designer of an Application-Specific Integrated Circuit

could decide which functional units were suited the particular application, combine them with on chip ROM and RAM, and end up with a single IC tailored to the particular application.

11.4 The Future of Formal Methods in Hardware Design

Still in its infancy, there is much interest in the promised benefits of applying Formal Methods to hardware design. All manufactureers desire the first time correctness which Formal Methods try to supply. Designing with formal methods takes time, but the delays and costs of faulty designs are significant enought to justify their use. Currently there are still some problems preventing its widespread use as a design aid.

Few people are experienced in the techniques of hardware specification and proof. Although only a small number of computer scientists, computer architects and VLSI designers have the knowledge, one could always include mathematicians in a design team. These mathematicians can busy themselves with the proofs, while the the other members of the team do the implementation.

While it may be possible to specify the very complex designs at a high level, at lower levels the amount of information can easily exceed the capabilities of current techniques. To prove even simple designs requires a large amount of human and CPU time. Although proof systems and their automated tactics will undoubtedly improve in the future, fabrication technology will also permit even more complex designs than the current million transistors on a single integrated circuit. There may always be a perpetual gulf between designs which are proven reliable and those which are 'state of the art'

Even if complex microprocessors are beyond complete specification, parts of them will not be. For example, the microcode of the floating point unit of the Inmos T800 transputer was implemented against a **Z** specification of an IEEE floating point standard [**?**]. If standard components are fully specified then it will be easier to combine them together reliably. Formal Methods are unlikely to let just anyone design a high performance VLSI design, but may be the only tool possible to help the experienced designers produce ever more complex sytems.

There is a need for reliable systems wherever computers are to be embedded into systems where failure could cost lives, from cars and medical equipment to aeroplanes and power stations. While Formal Methods can not guarantee that these systems will work, they are the first step to making VLSI design as safe and sound as the traditional fields of engineering.

Appendix A

Acknowledgements

A.1 People

The contributions of the following people should be acknowledged; they are credited in alphabetical order.

John Dow
Component Purchasing.

Mike Fourman
LFCS
Introducing me to Lambda.

Archie Howitt
Project Lab. Supervisor
Warning me off using BEPI.

The Techs

Department of Computer Science Rapid Deployment Force

Books, wires & various pieces of equipment.

Nigel Topham

Project Supervisor

Suggesting the use of SSRs and a parellel port, shifting & zero detection via PALS. Not hassling our VLSI team too much about deadlines.

Rongvald Walls

Heriott Watt University; CERN/DD

Passing on the Ultimate RISC article in Computer Architecture News.

A.2 Kit

The following equipment was abused beyond the point of no return.

APM

68000 APM with modified Real Time Systems board. Used for hardware and software development.

LFCS Suns

Used for the formal specification.

CS Suns

Typesetting the final report using LaTeX.

Macintoshes

Producing the first two reports and overheads for presentations.

HP1650A Logic Analyser Getting my board to work.

Vectras

Designing the circuits using P-CAD; EPLD programming.

Coffee Room Vending Machines

An invaluable source of caffeine and consumables.

Appendix B

Components Used

Part	Quantity	Description
AM29818	21	Shadow Serial Register
74F157A	2	Multiplexer
74F381	12	ALU/ function generator
74F182	4	Carry Lookahead Generator
74F163A	4	4 bit counter
74F541	8	Tristate Buffers
74F253	1	Multiplexer
74F10	1	Nand gates
74F04	1	Inverters
74F353	1	Latches
EP600	2	EPLD
EP310	1	EPLD
PAL10H8	6	PAL
MCM6164-45	4	8K * 8 SRAM @ 45 nS
40 MHz Osc.	1	Clock Oscillator Module
0.1 μF Capacitors	68	IC decoupling
47 μF Capacitors	1	Board decoupling
LED	2	Light Emitting Diode
Resistors	3	
Eurocard	1	Board used for construction
Edge Connector	2	Connection to host

All the components except for the PALS can be re-used in other projects.

Appendix C

Construction

This appendix describes the actual construction of the Ultimate RISC. The layout of the board is detailed, followed by a list of construction problems.

C.1 Layout

Building a board to operate at high speed, it was of the utmost importance to minimise the lengths of connecting wires. This reduced the propagation delay and the likiehood of cross-talk or transmission line effects.

TTL devices can draw a large amount of power, and so each IC had to be decoupled with a small capacitor across the power and ground pins. The entire board also needed decoupling in the form of a larger capacitor across the power supply inputs.

The computer was designed to connect to a M6809 board within an APM cab-

inet. This fixed the construction area to that of a double size Eurocard. The wire wrap board provided subsurface power and ground planes to reduce noise and ease power distribution. With seventy ICs to place on this board careful packing was necessary.

Wire wrap sockets were used to hold all ICs. This enabled easy insertion and removal of devices and permitted wire wrapping.

The connection between the Ultimate RISC and the Real Time Board was via a Eurocard edge connector. This had to placed at the bottom rear of the board —as seen from the front of the cabinet— to enable it to be connected to the PIA ports. It was this fact which determined the layout of the rest of the board (figure **??**).

The host inputs had to be latched and then passed to the Control Unit, which had therefore to be adjacent to the connector and latches. The common clock was a highly critical signal, and all devices which received it needed to be as close together as possible. This placed the PC, the address decoder and the clock oscillator all near to the Control Unit.

The Execution Unit consisted mainly of sixteen registers. These were easiest lain out together in a large block, achieving high density and aiding wiring of the SSR chain. The unit most tightly coupled to the Control Unit, it was placed immediately above this EPLD. The operand index addition was performed by a row of ICs above these registers.

The two other units needing placement were the ALU and the RAM. The RAM was only four, albeit large, chips, and were easily fitted into a spare area at the base of the board, next to the address decoder.

The ALU was allocated the top third of the card, with its interface to the data

Figure C.1: Board Layout

bus at the base of this region. The longest distance which signals travel to reach this unit is approximately 15cm, which should cause an extra delay of under a nanosecond.

Two light emitting diodes were placed above the execution unit, where they are easily visible. These provide feedback as to the operation of the board, lighting up when power is supplied and flickering during operation. The top LED emits a green light whenever the board is not halted. The lower LED is connected to the clock signal. It flashes whenever data is being transferred between this boad and the host, and is bright red during instruction execution.

A second edge connector was inserted into the board. This was purely to hold the board in place while being tested on an extender card.

C.2 Power Supply

The Ultimate RISC draws four Amps of current, a power consumption of 20 Watts. This is mainly due to the requirements of the AMD registers, which contain ECL logic internally. This power has to be dissipated by these devices, which can become almost too hot to touch. The power consumption is too large to be supplied by the single thin wire from the 6809 board. Instead the Ultimate RISC has to be connected to an external power supply. This also provides a convenient method of resetting the computer. It also created a few unforseen problems.

C.3 Construction Problems

The following problems all caused delays during construction.

Faulty EPLD programming

Mistakes made in the design of the EPLD programs for the Control Unit and address decoder caused regular problems. Some mistakes caused the computer to behave unreliably, or only partly correctly. For example, an early version of the control unit moved the source to source, an operation which is not of much use. The EPLD Software also lacked reliability. This is most apparent when using the state machine design program. This program ignores the definition of output transitions for a particular state, and yet will not produce a state machine at all if this state is not included. I was forced to retain this state but bypass it during instruction execution. Sometimes, however, the EPLD suffers from metastability problems, and in these situations the state can be reached; a power down reset is then obligatory.

Faulty Wiring

A few wrongly connected wires created difficulties. An accidental short of the output enable control of the Data register prevented memory read accesses from working for a whole week. Mis-wired power and ground connections on an IC also melted a buffer IC; a replacement was easily obtained.

Power Supply Difficulties

At one stage I was confused for a number of days by the nondeterministic behaviour of the computer. I suspected this was due to EPLD problems, and wasted much time trying to verify their operation. In fact, as was pointed out to me, an overloaded power supply was only producing a potential of 3.7 Volts between power and ground.

Upgrading to a higher rated power supply solved this problem, but created a new one. One one occasion this power supply had been switched on while repeated attempts were made to boot up an unreliable APM. A current loop must have been created as the power supply attempted to drive the entire APM via the ICs on my computer. I noticed the problem by the smell of solder, and then proceeded to burn my hand upon an AMD SSR. Suprisingly, these registers still worked later.

Avoiding Construction Problems

It is worthwhile studying these problems to see if the application of formal methods on a larger scale could have prevented them.

Mistakes in the EPLD programs which I had made myself could have been eliminated if the operation of the computer at this microstate level had been specified and verifed against the higher level specificatio. It could not prevent the problems caused by the EPLD programmer itself. If there was enough confidence in the state machine specifications, then perhaps these could have been implemented in a PAL instead.

Faulty wiring can be prevented by having a machine which can reliably wire

up a correct design. There do not seem to be one of these available within the department; even the BEPI robots's products have to be checked.

Formal Methods would not have provided any defence against the power supply problems; the devices were operating outside their stated limits. If I had had more confidence in my state machine design I may have come to suspect the power supply much earlier.

Note that there were no problems with the construction of the those parts of the ALU which were fully specified —the mathematics seem to be as predicted.

Appendix D

Using the Monitor Program

This appendix describes how to to use the monitor program to control the Ultimate RISC.

To make use of the monitor it is best to have read the main body of the project report, paying special attention the the host interface.

One should also consult the CS department report on using the M6809 monitor [**?**].

D.1 Main Points

This monitor program is derived from the 6809 monitor, as it uses this processor to communicate with the Ultimate RISC. Some of the basic 6809 commands are still available, although renamed.

The commands it supports range from high level commands to load and run programs on the Ultimate RISC, down to basic signal manipulation.

While the monitor examination of the Ultimate RISC's registers, these are only a copy of the actual registers. An explicit command to get an updated copy of these registers must be issued whenever the current state is required.

D.2 Files

The monitor consists of a number of files stored in the directory '**c::sal**'. The main 68000 based program is titled **monitor**. The 6809 Virtual PIA program is stored in the file **pia5.obj**. Access to these two programs, and the two files **registers** and **states**, is required to use the monitor.

D.3 Using the monitor

An APM with a 6809 board and Wyse terminal is a prerequisite to running this program. To start using the monitor issue the command

```
sal:monitor
```

After a short pause the monitor screen display will appear. This comprises a status display at the top of the screen with a command region below . One status line shows the current state of the Control Unit as a mnemonic name. The line beneath this gives the state of all the I/O lines; a signal is 'true' when its name appears in inverse video. This display is updated after every command.

The command **help** will list all other available commands.

To stop the monitor type either **quit** or control-Y.

D.4 States

The Control Unit can be in any one of thirty-two states. Only about half of these are actually used, and the unit should never enter any of the other states. The most important states are listed below.

Halted

This is the power up state, also entered after a memory read is prevented by the memory unit.

Waiting

The control unit is ready and waiting for use. It can be instructed to:-

- read and write registers
- read and write memory
- execute instructions

No Board!

The monitor can not detect the presence of the Ultimate RISC. It may not be installed or powered up.

Memory Access States

Any state with the name **read-X** or **write-X** is part of the host memory read or write cycles. These should not be encountered during normal use.

Unknown States

Any state named **unknown-XX** is not part of the programmed state machine. If any of these states appear then it is probably due to a metastability problem; if a known state can not be reached then a power down reset is obligatory.

Instruction Execution

Other named states are steps in the process of instruction execution.

D.5 Basic Commands

Register Manipulation

The monitor's copy of the registers can be listed with the command **registers**. This lists each register name with its value in hexadecimal. To update this copy

with the Ultimate RISC's current state, use the command **get**. This lists the registers after updating them.

Memory Access

A region of memory can be examined by the command:-

```
read <from> <to>
```

This gives a list of the contents of the memory locations between the two addresses. Any memory location with a horizontal line in place of a numeric value for the contents is a write only address.

To change an address in memory use the command

```
write <address> <value>
```

To download a file into memory, using the format listed in this report, issue the command

```
download <filename>
```

Instruction Execution

Instruction execution can be controlled either by the host or the on board clock. To control instructions from the host issue the command

```
execute <count>
```

The specified number of instructions will be executed, unless a halt state is reached earlier. If a zero count is given then instructions will be executed until halted. A count of the number of instructions executed is provided afterwards.

To let the Ultimate RISC execute instructions at full speed, issue the command **spin**.

To stop instruction execution at any time press control-C.

D.6 Low Level Commands

Signal Manipulation

The output signals can be directly toggled by entering the signal name followed by a zero or a one. This updates the state display, but the changes are not sent to the ultimate RISC until explicitly transmitted with the command **tx**.

Clock Control

A single clock pulse can be transmitted to the Ultimate RISC by the command **step**. To place the Ultimate RISC into freerunning mode clear the **freerun** signal and transmit the change.

Low Level Register Manipulation

To alter the value of a register, use the command:-

```
set <register> <value>
```

Only the address and data registers are currently reloaded, although the control unit can be reprogrammed to load the instruction register as well. This copy must be shifted back to the shadow registers by the command **put**. The Control Unit must then be instructed to load the address and data registers from their shadows; the command **store** attempts to do this.

Memory Testing

Two commands are provided to test the correct operation of memory. They both write a pattern of bits into a memory region, then read back result to log any differences. This is useful in testing for the correct operation of both the control unit and memory.

To test an arbitrary region of memory use the command:-

```
memtest <from> <to> <pattern>
```

This writes the pattern into all the memory locations, unless the pattern is equal to the start address. In this special case the pattern is incremented so it is always equal to the location written to. This provides an effective test for folded memory locations.

An extended test of RAM is also available:-

```
soak <repetitions>,<failures>
```

This repeatedly tests RAM with patterns of alternate bits and addresses until either the specified number of repetitions has been completed or the stated

number of failures has been exceeded. If either of the the two parameters is zero then that parameter is ignored —either repeating indefinately or ignoring the number of failures.

6809 commands

Most of the 6809 commands have been removed, with only four still supported. They are:-

obj download object code file

run execute a program (was **go**)

dump dump memory region

byte write a byte to memory

If one is experimenting with 6809 programs the command **initialise** is useful; this attempts to reinitialise the PIA port and common variables.

D.7 Maintenance

The IMP source code for the monitor is in the file '**sal:monitor**'. The 6809 assembly language program is in '**sal:pia5.asm**'. There are also two files to describe the states and the registers.

D.7.1 States

This file contains thirty-two lines, each listing the mnemonic name for a state. It should be updated with every reprogramming of the control EPLD.

D.7.2 Registers

This file records the name of each register and their positions with the SSR chain. Each register IC is only eight bits wide, so a thirty two bit register is built out of four SSRS. These sub-registers do not have to be adjacent to one another in the SSR chain.

The file starts with a number listing the number of registers in the chain, followed by the list of sub-registers, one to a line. Each sub-register is listed with the name of the register and the byte within that register which it is, in the range 0 to 3. The order of the registers in the file is that of the SSR chain, with the register connected to the host's SDI input first in this list.

D.8 Error Messages

There are four messages which the monitor issues, indicating something is not quite right with the Ultimate RISC. They may be followed with some technical

detail indicating what the monitor was trying to do, and what the response is.

```
'0' not passed through SSR chain
              or
'1' not passed through SSR chain
```

These messages state that control signals have not propagated all the way through the chain of registers. The response to either of these should follow the following sequence:-

1. try to **get** or **put** the registers a few more times
2. check the board is powered up and plugged in
3. check the number of registers installed matched that stated in the file 'Registers'
4. push all the registers firmly into their sockets

```
Not in the right state!
```

The control unit is not in the correct state for the required operation —usually involving a memory access. Try issuing the **step** command a few times to see if the waiting or halted states can be reached. If the step command has no effect then the board must be reset.

```
Attempted SSR writeback!
```

This error should never occur in normal use, unless manually manipulating the output lines. The current control signals, if transmitted, could have caused the

SSR registers to try driving their inputs —a dangerous operation which has been automatically intercepted. Consult the AMD databook to see why this should be avoided [**?**].

Appendix E

EPLD and PAL Programs

These are the programs used in the different EPLDs and PALS in the computer.

E.1 Control EPLD Program

This is the Moore Machine to control instruction execution and memory accesses. It can fit upon an EP610 or EP600. The state machine is illustrated in figure **??**.

```
Steve Loughran
Edinburgh University
29/3/89
1
B
```

```
EP600
The Control EPLD -v3.0
OPTIONS:TURBO = ON
PART:EP600
INPUTS:
        %clock%
        CLK@1
%go run- active low%
        GO@11
%load registers%
        LOAD@23
%l/s - select read or write operation%
        LS@14

%halt - sent from the address decoders%
        HALT@2
OUTPUTS:

%read/write line%
        RW@10

%memory select - active high%
        MS@21



%load Memory Address Register%
        LOADM@6

%load Data register%
```

```
        LOADD@4

%load Instruction register%
        LOADI@20

%plus of course all the state outputs%
        STATE0@19
        STATE1@18
        STATE2@17
        STATE3@16
        STATE4@15

%source/destination select%
        SOURCE@9

%Increment the PC%
        INCPC@7

%load the SSR Pc register%
        LOADPCR@5

%operand output enable%
        OEOP@22


NETWORK:
        %Define the outputs%
        RW=CONF(RWc,)
        MS=CONF(MSc,)
```

```
        LOADM=CONF(LOADMc,)
        LOADI=CONF(LOADIc,)
        LOADD=CONF(LOADDc,)
        SOURCE=CONF(SOURCEc,)
        INCPC=CONF(INCPCc,)
        LOADPCR=CONF(LOADPCRc,)
        OEOP=CONF(OEOPc,)


MACHINE:        control
CLOCK:        CLK

STATES: [STATE4 STATE3 STATE2 STATE1 STATE0]
PU [0  0 0 0 0  ]
S1 [0 0 0 0 1  ]
S2 [0 0 0 1 0  ]
S3 [0 0 0 1 1  ]
S4 [0 0 1 0 0  ]
S5 [0 0 1 0 1  ]
S6 [0 0 1 1 0  ]
S7 [0 0 1 1 1  ]
S8 [0 1 0 0 0  ]
S9 [0 1 0 0 1  ]
S10 [0 1 0 1 0  ]
I0 [1 0 0 0 0  ]
I1 [1 0 0 0 1  ]
I2 [1 0 0 1 0  ]
I3 [1 0 0 1 1  ]
I4 [1 0 1 0 0  ]
```

```
I5 [1 0 1 0 1  ]
I6 [1 1 1 0 0  ]
I7 [1 1 1 0 1  ]

PU:
        IF GO * LOAD THEN S1

S1:
        IF /LOAD THEN S2
        IF /GO THEN I0

S2:
        IF /LOAD THEN S2
        IF /GO THEN S1
        IF LS THEN S3
        S7

S3:        IF /HALT THEN PU
        S4

S4:        IF /HALT THEN PU
        S5

S5:     IF /HALT THEN PU
        S6

S6:     IF /HALT THEN PU
                S1
```

```
S7:     %start write%
        S9

%the next state should never be reached; the state machine programmer does not
recognise the exit transition, but will not compile without it...%

S8:      %dead state%
        S0

S9:     %keep writing; assert MS%
        S10

S10:        %stop writing%
        S1

% Instruction Execution %

I0:        %Get PC, continue write%
        IF GO THEN S1
        I1

I1:        %PC out%
        I2

I2:        %I-fetch & increment PC%
        IF /HALT THEN PU
        I3

I3:        %I strobe%
```

```
        I4

I4:        %source fetch%
        I5

I5:        %source strobe%
        IF /HALT THEN PU
        I6

I6:        %destination out%
        I7

I7:        %write to destination%
        I0


%now give the  signal outputs as a truth table%

T_TAB: STATE4 STATE3 STATE2 STATE1 STATE0:
        RWc MSc LOADMc LOADDc LOADIc LOADPCRc INCPCc OEOPc SOURCEc;
     %halted%
%PU%  0 0 0 0 0: 1 1 0 0 0 0 0 1 0;
     %waiting%
%S1%  0 0 0 0 1: 0 0 0 0 0 0 0 1 0;
     %loaded%
%S2%  0 0 0 1 0: 1 0 1 1 0 0 0 1 0;
     %read-1%
%S3%  0 0 0 1 1: 1 1 0 0 0 0 0 1 0;
     %read-2%
```

```
%S4%  0 0 1 0 0: 1 1 0 0 0 0 0 1 0;
     %read-3%
%S5%  0 0 1 0 1: 1 1 0 0 0 0 0 1 0;
     %read-end%
%S6%  0 0 1 1 0: 1 0 0 1 0 0 0 1 0;

     %write-1%
%S7%  0 0 1 1 1: 0 0 0 0 0 0 0 1 0;

% no S8 outputs given%

     %write-2%
%S9%  0 1 0 0 1: 0 1 0 0 0 0 0 1 0;
     %write-end%
%S10% 0 1 0 1 0: 0 0 0 0 0 0 0 1 0;


%Instruction execution signals%

     %get pc%
     %pc-addr%
%I0%  1 0 0 0 0: 0 0 0 0 0 1 0 1 0;
     %i-fetch (output PC)%
%I1%  1 0 0 0 1: 1 0 1 0 0 0 0 1 0;
     %i-strobe%
%I2%  1 0 0 1 0: 1 1 0 0 0 0 0 0 0;
     %s-addr (calculate source address)
%I3%  1 0 0 1 1: 1 1 0 0 1 0 1 0 0;
     %s-fetch%
```

```
%I4%  1 0 1 0 0: 1 1 1 0 0 0 0 0 1;
     %s-strobe%
%I5%  1 0 1 0 1: 1 1 0 0 0 0 0 0 1;
     %d-addr%
%I6%  1 1 1 0 0: 0 0 1 1 0 0 0 1 0;
     %d-write%
%I7%  1 1 1 0 1: 0 1 0 0 0 0 0 0 0;
END$
```

Figure E.1: Control Unit State Transition Diagram

E.2 Address Decoders

The address decoder is implemented in two EPLDS. At the slow speeds of my prototype, both could in fact be combined into the single EP310 EPLD. If the control EPLD was ever upgraded to an EP610, then the complex address decoder would also have to be upgraded to one which could operate at the same clock speed, which an EP310 or EP320 can not do.

E.2.1 Complex Address Decoder

This EP600 program decodes the following signals:-

- the control of the source of the PC increment signal. The increment signal should only be connected to the data bus for one clock cycle, when the skip register is written to. This increment must be done before the PC is loaded into the PCR or the skip is delayed.
- the loading of the CC register after a write to the Carry.
- loading the CC and the ACC registers after an ALU operation. This must be done at least 100 nS after the start of the write cycle.

To control the timing of signal issue, a Mealy Machine is used.

```
Steve Loughran
Edinburgh University
30/3/89
1
A
```

```
EP600
Complex Address Decoder
OPTION:TURBO=ON
PART:EP600
INPUTS:
        %clock%
        CLK@1
        %control signals%
        MS@2
        RW@23
        %address lines%
        A14@6
        A4@7
        A3@5
        A1@10
        A0@11

OUTPUTS:
%state outputs%
        C0@9,C1@8,C2@4
%load register signals%
        LOADA@15
        LOADCC@16
%select source of PC count instruction%
        SKIP@17

NETWORK:
        SKIP=CONF(SKIPc,)
```

```
MACHINE: complex
CLOCK: CLK
STATES: [C2     C1 C0 LOADA LOADCC]
PU [0 0 0 0 0]
S0 [1 1 1 0 0]
S2 [0 1 0 0 0]
S3 [0 1 1 0 0]
S5 [1 0 1 1 1]
S6 [1 1 0 0 1]

PU:
        S0
S0:
        %skip%
        IF MS*/A14*/A4*/A3*/A1*A0*/RW THEN S2
        %alu operation%
        IF MS*/A14*/RW*A4 THEN S3
        %load Carry%
        IF MS*/RW*/A14*/A4*A3 THEN S6

        %enable count from data bus0 for one cycle only%
        OUTPUTS:
                IF MS*/A14*/A4*/A3*/A1*A0*/RW THEN SKIPc
S1:
        S2
S2:         %wait till end of write cycle%
        IF /MS + RW THEN S0

        %perform an ALU operation%
```

```
S3:
        S5
S5:           %wait till end of cycle%
        IF /MS +RW THEN S0

                %load condition code register%
S6:           %wait till end of write cycle%
        IF /MS +RW THEN S0
END$
```

E.2.2 Simple Address Decoder

The remaining EPLD addresses are all implemented in a purely combinatorial manner.

```
Steve Loughran
Edinburgh University
30/3/89
1
A
EP310
Simple Address Decoder
OPTION: TURBO=ON
PART: EP310
INPUTS:
        CLK@1
        % control signals%
        RW@3
        MS@2
```

```
        %address lines%
        A0@9
        A1@8
        A3@6
        A4@5
        A14@4
OUTPUTS:
        HALT@19
        LOADX@18
        LOADY@17
        LOADPC@16
        OEACC@15
        OECC@14
NETWORK:
        CLK=INP(CLK)
        RW=INP(RW)
        MS=INP(MS)
        A0=INP(A0)
        A1=INP(A1)
        A3=INP(A3)
        A4=INP(A4)
        A14=INP(A14)
        HALT=CONF(HALTc,)
        LOADX=CONF(LOADXc,)
        LOADY=CONF(LOADYc,)
        LOADPC=CONF(LOADPCc,)
        OEACC=CONF(OEACCc,)
        OECC=CONF(OECCc,)
EQUATIONS:
```

```
%the halt signal%
        HALTc=/(MS*/RW * /A14 * /A4 * /A3);
%output enables%
        OEACCc=/(MS * RW * /A14 * /A4 * /A3);
        OECCc=/(MS * /RW * /A14 * A4);
%register loading%
        LOADPCc=MS * /RW * /A14 * /A4 */A3 */A1 */A0;
        LOADXc=MS * /RW * /A14 * /A4 * /A3 */A1 */A0;
        LOADYc=MS */RW * /A14 * /A4 */A3 *A1 * A0;
END$
```

E.3 ALU shift and condition code programs

The ALU shift unit is implemented with four PALS identically programmed to shift seven bits a single bit to the right, and a special fifth device to shift the most significant bits. Currently this most significant shift is performed in an EPLD, rather than a PAL. The Condition Code evaluation is also done by an EPLD. This was because some spare EPLDs were available for use. By using these instead of PALS I avoided having to irreparably program two PALS. This has increased the propagation delay of the ALU by 20 nS; the PALs need to be programmed to achieve full speed.

E.3.1 Shift PALS

The first four PALS were all programmed with the PAL software on the APMS. This uses two files, a pin specification file and an equation specification file.

Pinout

```
{shift1.pin}
{the pinouts for the first four shift & zero detect PALS}

1 F0
2 F1
3 F2
4 F3
5 F4
6 F5
7 F6
8 F7
9 SHIFT
0
12 Z
19 H0
18 H1
17 H2
16 H3
15 H4
14 H5
13 H6
0
```

equations

```
{Stephen Loughran}
{Edinburgh University CS4}
```

```
{15/5/89}
{PAL 10H8}
{ALU Shift Pal -1}
{Pal Implementation}
MODE EQUIN
IN F0,F1,F2,F3,F4,F5,F6,F7,SHIFT

{each output is the input if shift is false}
{or the input to the left if the shift signal is true}

        H0=F0.\SHIFT+F1.SHIFT
        H1=F1.\SHIFT+F2.SHIFT
        H2=F2.\SHIFT+F3.SHIFT
        H3=F3.\SHIFT+F4.SHIFT
        H4=F4.\SHIFT+F5.SHIFT
        H5=F5.\SHIFT+F6.SHIFT
        H6=F6.\SHIFT+F7.SHIFT

        {The zero flag normally tests inputs 0-6 for being zero}
        {returning true if so}
        {but on a shift tests bits 1-7 instead}
Z=\SHIFT.\F0.\F1.\F2.\F3.\F4.\F5.\F6+SHIFT.\F1.\F2.\F3.\F4.\F5.\F6.\F7
OUT Z,H6,H5,H4,H3,H2,H1,H0
```

E.3.2 The Shift EPLD

This EPLD program shifts the most significant four bits of the result and the carry flag. It differs from the previous shift PALS in that the number of bits

being tested for zero is less —the carry flag is not included in any test.

```
Stephen Loughran
Edinburgh University CS4
15/5/89
1
A
EP310
ALU Shift Pal -2   Most Significant Bits

OPTION: TURBO=ON
PART: EP310
INPUTS:
        F28@1
        F29@2
        F30@3
        F31@4
        Cin@5
        F0@6
        SHIFT@9
OUTPUTS:
        Z@12
        CARRY@15
        H31@16
        H30@17
        H29@18
        H28@19
NETWORK:
        F28=INP(F28)
```

```
        F29=INP(F29)
        F30=INP(F30)
        F31=INP(F31)
        Cin=INP(Cin)
        F0=INP(F0)
        SHIFT=INP(SHIFT)

        H28=CONF(H28c,)
        H29=CONF(H29c,)
        H30=CONF(H30c,)
        H31=CONF(H31c,)
        CARRY=CONF(CARRYc,)
        Z=CONF(Zc,)

EQUATIONS:
        H28c=F28*/SHIFT+F29*SHIFT;
        H29c=F29*/SHIFT+F30*SHIFT;
        H30c=F30*/SHIFT+F31*SHIFT;
        H31c=F31*/SHIFT+Cin*SHIFT;
        CARRYc=Cin*/SHIFT+F0*SHIFT;
        Zc=/SHIFT*/F28*/F29*/F30*/F31+
            SHIFT*/F29*/F30*/F31*/Cin;
END$
```

E.3.3 Condition Code EPLD

This EPLD generates the zero and carry flags for the condition code register. The Zero flag is true when all five zero flags from the Shift Unit are true. The

carry flag is selected from the result of the shift unit or bit zero of the data bus, depending upon the state of address bus line 4. In this way the carry flag can be set by an explicit write operation.

```
Steve Loughran
Edinburgh University CS4
13/5/89
2
A
EP310
Condition Code PAL/EPLD
OPTION:TURBO=ON
PART:EP310
INPUTS:
        ADDR4@3
        Z0@4
        Z1@5
        DATA0@6
        Z2@7
        Z3@8
        Z4@9
        CIN@11
OUTPUTS:
        ZERO@19
        CARRY@17
NETWORK:
        ADDR4=INP(ADDR4)
        Z0=INP(Z0)
        Z1=INP(Z1)
```

```
        Z2=INP(Z2)
        Z3=INP(Z3)
        Z4=INP(Z4)
        DATA0=INP(DATA0)
        CIN=INP(CIN)
        ZERO=CONF(ZEROc,)
        CARRY=CONF(CARRYc,)
EQUATIONS:
        %Select carry from ALU or Databus%
        CARRYc=ADDR4*CIN+/ADDR4*DATA0;

        %calculate the Zero flag%
        ZEROc=Z0*Z1*Z2*Z3*Z4;
END$
```

Appendix F

The Formal Specification and Simulation

This appendix lists the second specification of the computer, along with the simulation derived from it. It is composed of a number of files, some of which the specification alone uses, some of which the simulation makes use of. The Lambda specification describes the operation of the ALU on a component by component basis, including timing constraints. The simulation is executable in ML, and describes the entire computer. As such it forms another specification, with the functions being described without too much implementation detail.

F.1 Lambda Specification

The lambda specification provides the core of the code for the simulation, but is too powerful to be compiled by ML. It can be used to some prove properties

of the computer. It can also be used to verify the speed of operation of the ALU. First defined are the thirty-two bit and fifteen bit boolean tuples used in the specification. Constants and conversion functions are also listed; these conversion functions do not work correctly in SML or New Jersey ML due to overflow conditions.

```
(* Datatypes.L v1.8
   ==================
Datatypes used in ultimate RISC: revised version
2/1/89 sal *)
(* Int15 *)
type Int15=(bool * bool * bool * bool * bool * bool * bool *bool
                * bool * bool * bool * bool * bool * bool * bool);
val Zero15=(false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,false):Int15;
val Maxint15=(true,true,true,true,true,true,true,
                true,true,true,true,true,true,true,true):Int15;
(* bit extraction functions *)
fun addressBit14 ((b,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit13 ((_,b,_,_,_,_,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit12 ((_,_,b,_,_,_,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit11 ((_,_,_,b,_,_,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit10 ((_,_,_,_,b,_,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit9  ((_,_,_,_,_,b,_,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit8  ((_,_,_,_,_,_,b,_,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit7  ((_,_,_,_,_,_,_,b,_,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit6  ((_,_,_,_,_,_,_,_,b,_,_,_,_,_,_):Int15)=b:bool;
fun addressBit5  ((_,_,_,_,_,_,_,_,_,b,_,_,_,_,_):Int15)=b:bool;
fun addressBit4  ((_,_,_,_,_,_,_,_,_,_,b,_,_,_,_):Int15)=b:bool;
```

```
fun addressBit3  ((_,_,_,_,_,_,_,_,_,_,_,_,b,_,_,_):Int15)=b:bool;
fun addressBit2  ((_,_,_,_,_,_,_,_,_,_,_,_,_,b,_,_):Int15)=b:bool;
fun addressBit1  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,b,_):Int15)=b:bool;
fun addressBit0  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,b):Int15)=b:bool;
(* a little type conversion utility *)
fun booltoNat false=0
 |  booltoNat true=1;
fun Int15toNat (i:Int15)=
        (booltoNat (addressBit0 i)) +
2 * (booltoNat (addressBit1 i)) +
4 * (booltoNat (addressBit2 i))
        + 8*(booltoNat (addressBit3 i)) +
16 * (booltoNat (addressBit4 i)) +
32 * (booltoNat (addressBit5 i))
        + 64*(booltoNat (addressBit6 i)) +
128*(booltoNat (addressBit7 i))
        + 256*(booltoNat (addressBit8 i)) +
512*(booltoNat (addressBit9 i))
        + 1024*(booltoNat (addressBit10 i)) +
2048*(booltoNat (addressBit11 i))
        + 4096*(booltoNat (addressBit12 i)) +
8192*(booltoNat (addressBit13 i))
        + 16384*(booltoNat (addressBit14 i));
(* Int32 -very similar to Int15 *)
type Int32=(bool * bool * bool * bool * bool * bool * bool *bool *
              bool * bool * bool * bool * bool * bool * bool * bool*
              bool * bool * bool * bool * bool * bool * bool *bool*
              bool * bool * bool * bool * bool * bool * bool *bool);
val Zero32=(false,false,false,false,false,false,false,false,
```

```
                false,false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,false):Int32;
val Maxint32=(true,true,true,true,true,true,true,true,
                true,true,true,true,true,true,true,true,
                true,true,true,true,true,true,true,true,
                true,true,true,true,true,true,true,true):Int32;
(* bit extraction functions *)
fun dataBit31 ((b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit30 ((_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit29 ((_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit28 ((_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit27 ((_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit26 ((_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit25 ((_,_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit24 ((_,_,_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit23 ((_,_,_,_,_,_,_,_,b,_,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit22 ((_,_,_,_,_,_,_,_,_,b,_,_,_,_,_,_,_,
                 _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit21 ((_,_,_,_,_,_,_,_,_,_,b,_,_,_,_,_,_,
```

```
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit20 ((_,_,_,_,_,_,_,_,_,_,_,_,b,_,_,_,_,
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit19 ((_,_,_,_,_,_,_,_,_,_,_,_,_,b,_,_,_,
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit18 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,b,_,_,
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit17 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,b,_,
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit16 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,b,
                _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit15 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit14 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
 _,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit13 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit12 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit11 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit10 ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit9  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit8  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
fun dataBit7  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                _,_,_,_,_,_,_,_,b,_,_,_,_,_,_,_,_,_,_,_):Int32)=b;
```

```
fun dataBit6  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,b,_,_,_,_,_,_):Int32)=b;
fun dataBit5  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,b,_,_,_,_,_):Int32)=b;
fun dataBit4  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,_,b,_,_,_,_):Int32)=b;
fun dataBit3  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,_,_,b,_,_,_):Int32)=b;
fun dataBit2  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,_,_,_,b,_,_):Int32)=b;
fun dataBit1  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,b,_):Int32)=b;
fun dataBit0  ((_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,
                  _,_,_,_,_,_,_,_,_,_,_,_,_,_,_,_,b):Int32)=b;
(* another little type conversion utility *)
fun Int32toNat i=
        (booltoNat ( dataBit0 i)) +
2 * (booltoNat (dataBit1 i)) +
4 * (booltoNat (dataBit2 i))
        + 8*(booltoNat (dataBit3 i)) +
16 * (booltoNat (dataBit4 i)) +
32 * (booltoNat (dataBit5 i))
        + 64*(booltoNat (dataBit6 i))
+ 128*(booltoNat (dataBit7 i))
        + 256*(booltoNat (dataBit8 i))
+ 512*(booltoNat (dataBit9 i))
        + 1024*(booltoNat (dataBit10 i))
+ 2048*(booltoNat (dataBit11 i))
        + 4096*(booltoNat (dataBit12 i))
```

```
+ 8192*(booltoNat (dataBit13 i))
        + 16384*(booltoNat (dataBit14 i))
+ 32768*(booltoNat (dataBit15 i))
        + 65536*(booltoNat (dataBit16 i))
+ 131072*(booltoNat (dataBit17 i))
        + 262144*(booltoNat (dataBit18 i))
+ 524288*(booltoNat (dataBit19 i))
        + 1048576*((booltoNat (dataBit20 i))
         + 2*((booltoNat (dataBit21 i))
          + 2*((booltoNat (dataBit22 i))
           + 2*((booltoNat (dataBit23 i))
            + 2*((booltoNat (dataBit24 i))
             + 2*((booltoNat (dataBit25 i))
              + 2*((booltoNat (dataBit26 i))
               + 2*((booltoNat (dataBit27 i))
                + 2*((booltoNat (dataBit28 i))
                 + 2*((booltoNat (dataBit29 i))
                  + 2*((booltoNat (dataBit30 i))
                   + 2*(booltoNat (dataBit31 i))
                  )
                 )
                )
               )
              )
             )
            )
           )
          )
         )
        )
```

```
       )
      ;

(* definition of 4-tuples and 8-tuples (used in the alu) *)
type 'a four_tuple = ('a * 'a * 'a * 'a);
type nibble=bool four_tuple;
fun one4 (_,_,_,a) = a;
fun two4 (_,_,a,_)=a;
fun three4 (_,a,_,_)=a;
fun four4 (a,_,_,_)=a;
type 'a eight_tuple = ('a * 'a * 'a * 'a * 'a * 'a * 'a * 'a);
(* and a function to convert a 32 bit number into an 8-tuple
of boolean 4-tuples *)
fun split d=
        ((dataBit31 d,dataBit30 d,dataBit29 d,dataBit28 d),
         (dataBit27 d,dataBit26 d,dataBit25 d,dataBit24 d),
         (dataBit23 d,dataBit22 d,dataBit21 d,dataBit20 d),
         (dataBit19 d,dataBit18 d,dataBit17 d,dataBit16 d),
         (dataBit15 d,dataBit14 d,dataBit13 d,dataBit12 d),
         (dataBit11 d,dataBit10 d,dataBit9 d,dataBit8 d),
         (dataBit7 d,dataBit6 d,dataBit5 d,dataBit4 d),
         (dataBit3 d,dataBit2 d,dataBit1 d,dataBit0 d));
type byte=bool eight_tuple;
fun split8 d=
        ((dataBit31 d,dataBit30 d,dataBit29 d,dataBit28 d,
         dataBit27 d,dataBit26 d,dataBit25 d,dataBit24 d),
         (dataBit23 d,dataBit22 d,dataBit21 d,dataBit20 d,
         dataBit19 d,dataBit18 d,dataBit17 d,dataBit16 d),
         (dataBit15 d,dataBit14 d,dataBit13 d,dataBit12 d,
```

```
         dataBit11 d,dataBit10 d,dataBit9 d,dataBit8 d),
         (dataBit7 d,dataBit6 d,dataBit5 d,dataBit4 d,
         dataBit3 d,dataBit2 d,dataBit1 d,dataBit0 d));
fun Int15toByte ((a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0):Int15)=
        ((false,a6,a5,a4,a3,a2,a1,a0),(b7,b6,b5,b4,b3,b2,b1,b0));
(* inverse functions *)
fun BytetoInt15 (_,a6,a5,a4,a3,a2,a1,a0) (b7,b6,b5,b4,b3,b2,b1,b0)=
        (a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0);
fun merge8 (a7,a6,a5,a4,a3,a2,a1,a0) (b7,b6,b5,b4,b3,b2,b1,b0)
           (f7,f6,f5,f4,f3,f2,f1,f0) (g7,g6,g5,g4,g3,g2,g1,g0)=
        ((a7,a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0,
         f7,f6,f5,f4,f3,f2,f1,f0,g7,g6,g5,g4,g3,g2,g1,g0):Int32);
fun merge  (a7,a6,a5,a4) (a3,a2,a1,a0) (b7,b6,b5,b4) (b3,b2,b1,b0)
           (f7,f6,f5,f4) (f3,f2,f1,f0) (g7,g6,g5,g4) (g3,g2,g1,g0)=
        ((a7,a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0,
         f7,f6,f5,f4,f3,f2,f1,f0,g7,g6,g5,g4,g3,g2,g1,g0):Int32);
fun splitInt15 ((a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0):Int15)=
        ((false,a6,a5,a4),(a3,a2,a1,a0),(b7,b6,b5,b4),(b3,b2,b1,b0));
fun mergeInt15 (_,a6,a5,a4) (a3,a2,a1,a0) (b7,b6,b5,b4) (b3,b2,b1,b0)=
        ((a6,a5,a4,a3,a2,a1,a0,b7,b6,b5,b4,b3,b2,b1,b0):Int15);
(* plus one to split & merge into4 tuples of 7 plus 4 leftovers *)
fun merge7 (h3,h2,h1,h0)
   (a6,a5,a4,a3,a2,a1,a0) (b6,b5,b4,b3,b2,b1,b0)
           (f6,f5,f4,f3,f2,f1,f0) (g6,g5,g4,g3,g2,g1,g0)
   =
        ((h3,h2,h1,h0,a6,a5,a4,a3,a2,a1,a0,b6,b5,b4,b3,b2,b1,b0,
         f6,f5,f4,f3,f2,f1,f0,g6,g5,g4,g3,g2,g1,g0):Int32);
fun split7 ((h3,h2,h1,h0,a6,a5,a4,a3,a2,a1,a0,b6,b5,b4,b3,b2,b1,b0,
         f6,f5,f4,f3,f2,f1,f0,g6,g5,g4,g3,g2,g1,g0):Int32)=
```

```
((h3,h2,h1,h0),
   (a6,a5,a4,a3,a2,a1,a0),(b6,b5,b4,b3,b2,b1,b0),
           (f6,f5,f4,f3,f2,f1,f0),(g6,g5,g4,g3,g2,g1,g0));
```

These functions extract the source destination and index fields for use in instruction execution.

```
(* Decode.L
   ========
        Functions which  extract portions  of a number for address decoding
purposes
2/1/89 sal *)
(* The bit indicating whether to index the source or not *)
val IndexX=dataBit31;
(* The source address *)
fun Source i=
        (dataBit30 i,dataBit29 i,dataBit28 i,dataBit27 i,
         dataBit26 i,dataBit25 i,dataBit24 i,dataBit23 i,
         dataBit22 i,dataBit21 i,dataBit20 i,dataBit19 i,
         dataBit18 i,dataBit17 i,dataBit16 i):Int15;
val IndexY=dataBit15;
fun Destination i=
        (dataBit14 i,dataBit13 i,dataBit12 i,dataBit11 i,
         dataBit10 i,dataBit9  i,dataBit8  i,dataBit7  i,
         dataBit6  i,dataBit5  i,dataBit4  i,dataBit3  i,
         dataBit2  i,dataBit1  i,dataBit0  i):Int15;
(* extract the l.s. 15 bits from a data word *)
val Truncate15=Destination;
(* expand a single bit to the full length of a data word *)
```

```
fun Expand b=(false,false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,false,
                false,false,false,false,false,false,false,b):Int32;
```

A few functions had to be added so that the following files could be parsed by either Lambda or ML. A different copy of this file is used in the simulation which can not use Church' s iota function.

```
(* Spec.L 1.4
   ======
Special stuff put in to the Lambda specification alone  to make it more ML like
10/1/89 sal *)
fun not a=~a;
(* definition of successor-15 function avoiding binary maths *)
fun S15 i =
        if i == Maxint15 then Zero15 else
                iota j. Int15toNat j == S(Int15toNat i);

(* definition of successor-32 function avoiding binary maths *)
fun S32 i =
        if i == Maxint32 then Zero32 else
                iota j. Int32toNat j == S(Int32toNat i);
(* define time as measured in nanoSeconds*)
val nS=Natural;
```

All variables have to be explicitly stated in Lambda, unlike ML.

```
(* variables.L
```

```
   ===========
Variables used in the specification  over  and  above  those  over and above
those normally provided
2/1/89 sal *)
(* generate signals *)
vbl G,g0,g1,g2,g3,g4,g5,g6,g7,G0,G1     :bool;
(* propagate signals *)
vbl P,p0,p1,p2,p3,p4,p5,p6,p7,P0,P1     :bool;
(* carry bits *)
vbl carry_in,carry_out,c3,c7,c11,c15,c19,c23,c27        :bool;
(* zero flags *)
vbl z0,z1,z2,z3,z4;
(* general use (mainly in alu ) variables *)
vbl a0,a1,a2,a3,a4,a5,a6,a7,addr4;
vbl b0,b1,b2,b3,b4,b5,b6,b7;
vbl data0
vbl f,f0,f1,f2,f3,f4,f5,f6,f7,f28,f29,f30,f31;
vbl g,h,h0,h1,h2,h3,h4,h5,h6,h7,h28,h29,h30,h31;
vbl s0,s1,s2;
vbl shift;
```

Each ALU operation is first described as a function operating between two four bit tuples and a carry flag. These are then combined into a single function to mimic the functionality of the ALU.

```
(* Math.L 1.5 5/24/89
   ======
Definition of Int32 Maths as performed by the ALU.
The operations are given as functions with no timing constraints.
```

```
3/1/89 sal
        *)
(* the exclusive or function *)
infix 6 xor;
fun a xor b= (a ||| b) && ~ (a && b);
(* A full adder using xor *)
fun full_add a b c=
        a xor b xor c;
(* how to add two bool 4 tuples using carry lookahead *)
fun add4 (a3,a2,a1,a0) (b3,b2,b1,b0) c=
  let   val g0=a0 && b0
        val g1=a1 && b1
        val g2=a2 && b2
        val p0=a0 ||| b0
        val p1=a1 ||| b1
        val p2=a2 ||| b2
  in
        (full_add a3 b3 (g2 ||| (p2 && (g1 |||
                                (p1 && (g0 ||| (p0 && c))))))),
         full_add a2 b2 (g1 ||| (p1 && (g0 ||| (p0 && c)))),
         full_add a1 b1 (g0 ||| (p0 && c)),
 full_add a0 b0 c)
           end;
fun preset4 _ _ _=(true,true,true,true);
fun clear4  _ _ _=(false,false,false,false);
fun and4 (a3,a2,a1,a0) (b3,b2,b1,b0) _ =
        (a3 && b3,a2 && b2, a1 && b1,a0 && b0);
fun or4 (a3,a2,a1,a0)  (b3,b2,b1,b0)_ =
        (a3 ||| b3,a2 ||| b2, a1 ||| b1,a0 ||| b0);
```

```
fun xor4 (a3,a2,a1,a0) (b3,b2,b1,b0) _ =
        (a3 xor b3,a2 xor b2, a1 xor b1,a0 xor b0);

fun not4 (a3,a2,a1,a0) = (~a3,~a2,~a1,~a0);

(* dont know exactly what this does yet *)

fun minus4 a b c=
        add4 a (not4 b) c;
(* describe how a different selction of the alu function produces
   different results *)
fun applyALU a b c false false false= clear4  a b c
 |  applyALU a b c false false true = minus4  a b c
 |  applyALU a b c false true  false= minus4  b a c
 |  applyALU a b c false true  true = add4   a b c
 |  applyALU a b c true  false false= xor4    a b c
 |  applyALU a b c true  false true = or4     a b c
 |  applyALU a b c true  true  false= and4    a b c
 |  applyALU a b c true  true  true = preset4 a b c;
fun propagate (a3,a2,a1,a0) (b3,b2,b1,b0)=
        ~(a0 && b0 && a1 && b1 && a2 && b2 && a3 && b3);
fun generate (a3,a2,a1,a0) (b3,b2,b1,b0)=
  let   val g0=a0 && b0
        val g1=a1 && b1
        val g2=a2 && b2
        val g3=a3 && b3
        val p1=a1 ||| b1
        val p2=a2 ||| b2
        val p3=a3 ||| b3
```

```
in
      ~(g3 |||  p3 && g2 ||| p3 && p2 && g1
                        ||| p3 && p2 && p1 && g0)
end;
```

The components are individually specified as rewrite rules. The delays between inputs and outputs are specifed as constants, to simplify changing to different logic families.

```
    (* Components.L
    -the formal specification of individual components
     -with timing in nS
    2/1/89 sal: SN74F182 Look ahead carry unit
    3/1/89 sal: SN74F381 ALU
       *)
    (* SN74F182 Look-ahead carry generators
       ===================================
       These TTL  components take  as inputs  the P' and G' signals from
       either ALU's or other '182 units, to return a carry for each ALU/
       lookahead  unit,  and  Propagate  and  Generate  signals  for the
combined units *)
    (* delay between p,g signals and G *)
    val t_pg_G=8;
    (* delay between p,g signals and P *)
    val t_pg_P=6;
    (* delay between carry in and carries out *)
    val t_c_c=5;
    val t_pg_c=5;
    (* difference in delays between t_c_c and t_pg_c *)
```

```
val t_c_pg_c=t_c_c-t_pg_c;
val SN74F182 #(g0,g1,g2,g3,p0,p1,p2,p3,c,c1,c2,c3,G,P)=
(* g0-g3,p0-p3: generate and propagate inputs
   c: carry in
   c1,c2,c3: carry outputs
   G,P: Generate and Propagate outputs *)
   forall t:nS.
           (G (t+t_pg_G) == ~(g3 t || (p3 t && g2 t)
                   || (p3 t && p2 t && g1 t)
                   || (p3 t && p2 t && p1 t && g0)))
   /\
   forall t:nS.
           (P (t+t_pg_P) == ~(p0 t && p1 t && p2 t && p3 t))
   /\
   forall t:nS.
           (c1 (t+t_c_c) == g0 (t+t_c_pg_c) ||
                   (p0 (t+t_c_pg_c) && c t))
   /\
   forall t:nS.
           (c2 (t+t_c_c)== g1 (t+t_c_pg_c)||
                   (p1 (t+t_c_pg_c) && ((p0 (t+t_c_pg_c) && c t)
                           || g0 (t+t_c_pg_c))))
   /\
   forall t:nS.
           (c3 (t+t_c_pg_c)== g2 (t+t_c_pg_c)||
                   (p2 (t+t_c_pg_c) && (g1 (t+t_c_pg_c)||
                   (p1 (t+t_c_pg_c) && ((p0 (t+t_c_pg_c) && c t)
                           || g0 (t+t_c_pg_c)))));
```

```
(* SN74F381 : ALU/function generator
   ================================
          These are the TTL i.c's which form the  core of  the ALU;
   each takes  two 4-bit  words ,  carry in  and 3  bits to select a
   function. Seven possible functions  can  be  performed: addition,
   subtraction, preset,clear,  and, or, exclusive or. One can select
   which word is be subtracted from the other.
   The units produce propagate and generate signals for  use by '182
   look-ahead carry units. *)
(* typical propagation delays -from data sheet *)
val t_c_f=8;
val t_ab_g=7;
val t_ab_p=7;
val t_ab_f=11;
val t_s_any=10;
val t_ab_c_f=t_ab_f-t_c_f;
val SN74F381 #(a,b,c,p,g,s2,s1,s0,f)=
(* a,b:4-bit operands (input)
   c: carry in
   p,g:propagate and generate signals (output)
   s2,s1,s0: function select signals
   f: 4 bit result *)
   (a3,a2,a1,a0)=a /\
   forall t:nS.
          (p (t+t_ab_p)==propagate (a t) (b t))
   /\
   forall t:nS.
          (g (t+t_ab_g)==generate (a t) (b t))
   /\
```

```
        forall t:nS.
                (f (t+t_ab_f)==
                        applyALU (a t) (b t) (c (t+t_ab_c_f)
                                 (s2 t) (s1 t) (s0 t));
```

All the pals were defined with timing constraints included.


```
All the pals were defined with timing constraints included.
\begin{verbatim}
(* PALS.L
   ======
   Definition of PALS,EPLDS..etc.
   In ALU, address decoders, control...
3/1/89 sal :ALU function pals*)
(* delay for combinatorial PAL *)
val t_pal=25;
(* The programming for  four of the PALS in the Shift unit of the ALU *)
val ALU_shift_PAL #(shift,f7,(f6,f5,f4,f3,f2,f1,f0),
                z,(h6,h5,h4,h3,h2,h1,h0))=
        forall t:nS.
                (z (t+t_pal)==
                        (~(shift t) && ~(f0 t) && ~(f1 t) && ~(f2 t) &&
                        ~(f3 t) && ~(f4 t)  && ~(f5 t) && ~(f6 t))
                                 ||
                        (shift t && ~(f1 t) && ~(f2 t) && ~(f3 t) && ~(f4 t)
                        && ~(f5  t) && ~(f6 t) && ~(f7 t))
        /\
        forall t:nS.
```

```
        (h0 (t+t_pal)== (f0 t) && ~(shift t) ||
                        (f1 t) && (shift t))
/\
forall t:nS.
        (h1 (t+t_pal)== (f1 t) && ~(shift t) ||
                        (f2 t) && (shift t))
/\
forall t:nS.
        (h2 (t+t_pal)== (f2 t) && ~(shift t) ||
                        (f3 t) && (shift t))
/\
forall t:nS.
        (h3 (t+t_pal)== (f3 t) && ~(shift t) ||
                        (f4 t) && (shift t))
/\
forall t:nS.
        (h4 (t+t_pal)== (f4 t) && ~(shift t) ||
                        (f5 t) && (shift t))
/\
forall t:nS.
        (h5 (t+t_pal)== (f5 t) && ~(shift t) ||
                        (f6 t) && (shift t))
/\
forall t:nS.
        (h6 (t+t_pal)== (f6 t) && ~(shift t) ||
                        (f7 t) && (shift t));

(* the shift PAL program for the most significant PAL *)
```

```
          fun ALU_SHIFT_PAL_fn_2  shift d0 c f31 f30 f29 f28
           =
                let val h28= ( ~shift && f28 ||| shift && f29)
                and h29 = ~shift && f29 ||| shift && f30
                and h30 = ~shift && f30 ||| shift && f31
                and h31 = ~shift && f31 ||| shift && c
                and carry_out = ~shift && c ||| shift && d0
                and z = ~shift && ~f28 && ~f29 && ~f30 && ~f31
                        |||
                       shift && ~f29 && ~f30 && ~f31 && c
                in
                        (z,carry_out,(h31,h30,h29,h28))
                end;
val ALU_SHIFT_PAL_2 # shift d0 c f31 f30 f29 f28 z carry_out
               (h31,h30,h29,h28)=
        forall t:nS.
               (z (t+t_pal),carry_out (t+t_pal),
               (h31 (t+t_pal),h30 (t+t_pal),
               h29 (t+t_pal),h28 (t+t_pal)))==
               ALU_SHIFT_PAL_fn_2  shift (d0 t) (c t) (f31 t)
                                        (f30 t) (f29 t) (f28 t);
(* The Programming of the CC PAL in the ALU *)
val ALU_CC_PAL #(shift,z0,z1,z2,z3,z4,carry_in,data0,addr4
                z,c)=
  forall t:nS.
        (z (t+t_pal)== (z0 t) && (z1 t) && (z2 t) && (z3 t)&& (z4 t))
  /\
   forall t:nS.
        (c (t+t_pal)== (carry_in t) && (addr4 t) ||
```

```
                        (data0 t) && (~addr4 t));
```

The components can be combined to specify exactly how the ALU should behave. This could, if desired, be verified against the mathematics of of a subset of integers.

```
(*              ALU.L
                =====

The definition of the ALU as a whole.

3/1/89 sal: TTL part only; not the PALS
*)

(* The carry look-ahead unit *)
(* built out of three TTL devices *)

val carrylookahead #( g0,g1,g2,g3,g4,g5,g6,g7,
                      p0,p1,p2,p3,p4,p5,p6,p7,
                      c,c3,c7,c11,c15,c19,c23,c27,carry_out)=

        SN74F182 #(g0,g1,g2,g3,p0,p1,p2,p3,c,c3,c7,c11,G0,P0)
        /\
        SN74F182 #(g4,g5,g6,g7,p4,p5,p6,p7,c15,c19,c23,c27,G1,P1)
        /\
        SN74F182 #(G0,G1,true,true,P0,P1,true,true,c,c15,carry_out,
                      false,true,true);
```

```
(* the TTL portion of the ALU *)
(* eight four bit slices connected together via the carry generator *)
(* a= Input a
   b= Input b
   c= carry in
   s2-s0= control signals
   c=carry out
   f=evaluated function
    *)


val ttlALU # (a b c s2 s1 s0 carry_out f)=
        (a7,a6,a5,a4,a3,a2,a1,a0)==split a /\
        (b7,b6,b5,b4,b3,b2,b1,b0)==split b /\
        carrylookahead #( g0,g1,g2,g3,g4,g5,g6,g7,
                          p0,p1,p2,p3,p4,p5,p6,p7,
                          c,c3,c7,c11,c15,c19,c23,c27,carry_out) /\
        SN74F381#(a0,b0,c,p0,g0,s2,s1,s0,f0) /\
        SN74F381#(a1,b1,c3,p1,g1,s2,s1,s0,f1) /\
        SN74F381#(a2,b2,c7,p2,g2,s2,s1,s0,f2) /\
        SN74F381#(a3,b3,c11,p3,g3,s2,s1,s0,f3) /\
        SN74F381#(a4,b4,c15,p4,g4,s2,s1,s0,f4) /\
        SN74F381#(a5,b5,c19,p5,g5,s2,s1,s0,f5) /\
        SN74F381#(a6,b6,c23,p6,g6,s2,s1,s0,f6) /\
        SN74F381#(a7,b7,c27,p7,g7,s2,s1,s0,f7) /\
        split f=(f7,f6,f5,f4,f3,f2,f1,f0);
```

F.2 ML simulation

This extends the specification to describe instruction execution, and wraps this in a monitor shell. Some of the Lambda functions which only took a couple of lines have had to be translated into page-long ML operations. These implement a few lambda functions in ML. Not having an **iota** function, the successor functions are implemented by first defining functions converting from integers to Int15 and Int32. The successor function is implemented by converting the boolean tuple to an integer representation, adding one, then converting it back. The length of this file compared with the lambda specification indicates the differences in power of the two notations.

```
(* Extras.SIM  v1.9  5/24/89*)
(* Extra functions needed for the simulation that Lambda doesn't
   6/1/89 sal
   *)

          exception not_a_nibble;

          (*convert a number from  0-16 to a binary equivalent*)

          fun to_nibble n=
                if n>16 orelse n<0 then raise not_a_nibble
                else
                        ((n div 8) =1,
                         (n div 4 mod 2)=1,
                         (n div 2 mod 2)=1,
                         (n mod 2)=1);
```

```
(* convert any number to a list of nibbles *)

fun NattoNibbles n =
      if n<16 then [to_nibble n]
      else NattoNibbles (n div 16) @ [to_nibble (n mod 16)] ;

(* measure the length of a list *)

fun length nil = 0
 |  length (_::t) = (length t) + 1;

(* extend a list to the lenth required *)

exception list_too_long ;

fun extend n L= if (length L) < n
              then ((false,false,false,false)::extend (n-1) L)
              else
               if (length L) > n
               then raise list_too_long
               else L;


val maxint15=Int15toNat Maxint15;

exception address_too_big ;
```

```
          fun NattoInt15 n=
                if n<= maxint15 then
                        let val (a::b::c::d::_)= extend 4 ( NattoNibbles n)
in
(mergeInt15 a b c d):Int15
end
                else raise address_too_big;

  fun S15 word=
if word=Maxint15 then
Zero15
else
NattoInt15 (1 + (Int15toNat word));



          val maxint32=Int32toNat Maxint32;

          exception data_too_big ;

                fun NattoInt32 n=
                if n<=maxint32 then
let val [a,b,c,d,e,f,g,h]=extend 8 (NattoNibbles n )
in
merge a b c d e f g h
end
                else raise data_too_big;

          infix 8 &&;
```

```
          fun a && b = a andalso b;

          infix 7 |||;
          fun a ||| b = a orelse b;

          fun ~ a = not a;

fun plus15 a b=
NattoInt15 ((Int15toNat a) + (Int15toNat b));
```

All the register addresses are defined as constant 15-tuple addresses.

```
(*            CONSTANTS.SIM  v1.4 5/24/89
              =============  ===== ======

                Constant  values  for  use  by  the simulation: these
                consist of mnemonics for memory addresses
             6/1/88 sal *)
          (* program counter *)
          val PC=Zero15;

          (* skip register *)
          val SKIP=S15  PC;

          (* X index *)
          val X=S15  SKIP;

          (* Y index *)
          val Y=S15  X;
```

```
(* The number four in int15 format *)
val PLUS4= S15 o S15 o S15 o S15 ;

(*The accumulator on read *)
val ACC=NattoInt15 8;

(*which is the carry on write*)
val CARRYIN=ACC;

(*The accumulator Functions *)

val CLR=NattoInt15 16;
val SUBA=NattoInt15 17;
val SUBB=NattoInt15 18;
val ADD=NattoInt15 19;
val XOR=NattoInt15 20;
val OR=NattoInt15 21;
val AND=NattoInt15 22;
val SET=NattoInt15 23;

(* The additional value to cause a shift *)
val SHIFT=ACC;

(* the condition codes *)
val Z=CLR;
val N=SUBA;
val V=SUBB;
val CARRY=ADD;
```

The operating state of the Ultimate RISC is defined as a number of records. One gives the state of the execution unit, another that of the ALU. The entire machine is then represented as the two combined with a function to describe RAM.

```
(*  states.SIM v1.6
============

         (* State description for the simulation *)

          type EXstate={pc:Int15, x:Int15, y:Int15,halt:bool};

          type ALUstate={acc:Int32,z:bool,n:bool,v:bool,carry:bool};

          type State={mem:Int15->Int32,
exstate:EXstate,
alustate:ALUstate};

          (* functions to extract individual fields}
          fun get_mem ({mem=m,...}:State)=m;
          fun get_ex ({exstate=e,...}:State)=e;
          fun get_alu ({alustate=a,...}:State)=a;
          fun get_pc ({pc=p,...}:EXstate) =p;
          fun get_x ({x=x,...}:EXstate) =x;
          fun get_y ({y=y,...}:EXstate) =y;
  fun get_halt ({halt=h,...}:EXstate)=h;
          fun get_acc ({acc=a,...}:ALUstate) =a;
          fun get_carry ({carry=c,...}:ALUstate) =c;
          fun get_zero ({z=z,...}:ALUstate) =z;
```

```
          fun get_negative ({n=n,...}:ALUstate) =n;
          fun get_overflow ({v=v,...}:ALUstate) =v;
```

This describes the operation of the ALU. Based upon the lambda specification, the components have been redescribed as functions, and combined to describe the entire ALU's operation.

```
(* alu.SIM v1.8 *)
(* ======= =====*)

         (* Simulation of the ALU *)

(* the ALU bit slice component*)

          fun SN74F182 g0 g1 g2 g3  p0 p1 p2 p3 c= (* c1 c2 c3 G P *)
          let
                val G =(g3  && (p3  ||| g2 )
                               && (p3  ||| p2  ||| g1 )
                               && (p3  ||| p2 ||| p1  ||| g0))
                and  P=p0  ||| p1  ||| p2  ||| p3
                and  c1= not(g0 && (p1 ||| (~ c)))
                and  c2=not(g1 && (p0 ||| (g0 && p1 ||| (~ c))))
                and  c3= not(g2 && (p2 ||| (g1 && p0 |||
(g0 && p1 ||| (~ c)))))
          in
                (c1,c2,c3,G,P)
  end;

(* the lookahead carry generator *)
```

```
fun carrylookahead g0 g1 g2 g3 g4 g5 g6 g7
                  p0 p1 p2 p3 p4 p5 p6 p7 c=
let
      val (c3,c7,c11,G0,P0)=SN74F182 g0 g1 g2 g3 p0 p1 p2 p3 c
and    G1=(g7  && (p7  ||| g6 )
                       && (p7  ||| p6  ||| g5 )
                       && (p7  ||| p6 ||| p5  ||| g4))
and    P1=p4 ||| p5 ||| p6 ||| p7
in
      let
              val (c15,carry_out,_,_,_)=SN74F182 G0 G1 true true
                                                 P0 P1  true true
 c
      in
              let val (c19,c23,c27,_,_) = SN74F182 g4 g5 g6 g7
                                                   p4  p5  p6  p7
 c15
              in
                      (c3,c7,c11,c15,c19,c23,c27,carry_out)
              end
      end
end;


(* the TTL part of the ALU)
      fun ttlALU a b c s2 s1 s0=
      let
            val (a7,a6,a5,a4,a3,a2,a1,a0)=split a
```

```
and    (b7,b6,b5,b4,b3,b2,b1,b0)=split b
in
      let
              val (c3,c7,c11,c15,c19,c23,c27,carry_out)=
                      carrylookahead
                      (generate a0 b0)
                      (generate a1 b1)
                      (generate a2 b2)
                      (generate a3 b3)
                      (generate a4 b4)
                      (generate a5 b5)
                      (generate a6 b6)
                      (generate a7 b7)
                      (propagate a0 a0)
                      (propagate a1 a1)
                      (propagate a2 a2)
                      (propagate a3 a3)
                      (propagate a4 a4)
                      (propagate a5 a5)
                      (propagate a6 a6)
                      (propagate a7 a7)
                      c
      in (carry_out, merge
              (applyALU a7 b7 c27 s2 s1 s0)
              (applyALU a6 b6 c23 s2 s1 s0)
              (applyALU a5 b5 c19 s2 s1 s0)
              (applyALU a4 b4 c15 s2 s1 s0)
              (applyALU a3 b3 c11 s2 s1 s0)
              (applyALU a2 b2 c7 s2 s1 s0)
```

```
                    (applyALU a1 b1 c3 s2 s1 s0)
                    (applyALU a0 b0 c s2 s1 s0))
              end
        end;

(* the shift pal program for the four least significant PALS *)

        fun ALU_SHIFT_PAL_fn shift f7 (f6,f5,f4,f3,f2,f1,f0)=
        let val z= (~shift && ~f0 && ~f1 && ~f2 && ~f3 && ~f4
                                && ~f5 && ~f6 ) |||
                             (shift && ~f1 && ~f2 && ~f3 && ~f4 && ~f5
                               && ~f6 && ~f7 )
        and   h0= ~shift && f0 ||| shift && f1
        and   h1= ~shift && f1 ||| shift && f2
        and   h2= ~shift && f2 ||| shift && f3
        and   h3= ~shift && f3 ||| shift && f4
        and   h4= ~shift && f4 ||| shift && f5
        and   h5= ~shift && f5 ||| shift && f6
        and   h6= ~shift && f6 ||| shift && f7

        in
              (z,(h6,h5,h4,h3,h2,h1,h0))
        end;

(* the shift PAL program for the most significant PAL *)

  fun ALU_SHIFT_PAL_fn_2  shift d0 c f31 f30 f29 f28
   =
let val h28= ( ~shift && f28 ||| shift && f29)
```

```
and h29 = ~shift && f29 ||| shift && f30
and h30 = ~shift && f30 ||| shift && f31
and h31 = ~shift && f31 ||| shift && c
and carry_out = ~shift && c ||| shift && d0
and z = ~shift && ~f28 && ~f29 && ~f30 && ~f31
|||
       shift && ~f29 && ~f30 && ~f31 && c
in
(z,carry_out,(h31,h30,h29,h28))
end;



(* the condition code generation *)
          fun ALU_CC_PAL_fn shift z0 z1 z2 z3 z4 carry_in data0 addr4=
          let
                val z=z0 && z1 && z2 && z3 && z4
               and c=carry_in && addr4 ||| data0 && ~addr4
          in
                (z,c)
          end;

(* the complete ALU *)
(* takes the current ALU state, the data on the bus and the
value of the address bus to return an updated ALU state*)

          fun alu (alustate:ALUstate) d a=
          let
                val (c,f)=ttlALU (get_acc alustate)
                                 d
```

```
                                (get_carry alustate)
                                (addressBit2 a)
                                (addressBit1 a)
                                (addressBit0 a)
           and shift=addressBit3 a
          in
               let val ((f31,f30,f29,f28),f3,f2,f1,f0)=
split7 f in
                       let val (z0,h0)=ALU_SHIFT_PAL_fn shift (dataBit7 f) f0
                           and (z1,h1)=ALU_SHIFT_PAL_fn shift (dataBit14 f) f1
                           and (z2,h2)=ALU_SHIFT_PAL_fn shift (dataBit21 f) f2
    and (z3,h3)=ALU_SHIFT_PAL_fn shift (dataBit28 f) f3
                           and (z4,c2,(h31,h30,h29,h28))=
ALU_SHIFT_PAL_fn_2 shift
(dataBit0 f) c f31 f30 f29 f28
                       in
                               let val h = merge7 (h31,h30,h29,h28)
                                           h3 h2 h1 h0
                               in
                                       let val (z,carry)=
           ALU_CC_PAL_fn shift
                                              z0 z1 z2 z3 z4 c
(dataBit31 d)
(addressBit4 a)
                                       in
                                         ({acc=h,
                                          z=z,
                                          n=dataBit31 h,
                                          v=carry,
```

```
                                        carry=carry}:ALUstate)
                                    end
                                end
                            end
                        end
                    end;
```

The memory of the computer can be specified with some difficulty. RAM is described by a curried function. A number of boolean functions decode addresses and generate the halt signal. The read function returns either a RAM location's contents, the accumulator or a condition flag. The write function is more complex, being able to update the entire machine state.

```
(* memory.SIM v1.7  5/24/89 *)
(* ========== ==== ======= *)
(* Simulated memory functions *)
(* functional memory specification: very memory inefficient *)

    fun RAM (d:Int32) mem (a:Int15) aa=
        (* taken from Lambda's examples *)
            if a=aa then d else mem aa;

         (*reset state -not quite true*)
        fun RAM0 (_:Int15)=Zero32;

        (* A function to test if an address is that of ram or not
            i.e. returns True iff  bit 14 is True *)

        fun ram_address a=
```

```
      addressBit14  a;

(* A function to test if an address references the alu-
      i.e is in the range of addresses 16-31
      -or equivalently bit 4 =True and it is not a ram address *)

fun alu_address a=
      ~(ram_address  a) && (addressBit4  a);


(*  a function to check if an address is valid for a read.
a boolean value will be returned, true if an address is valid,
false otherwise.  This is  used to  intercept reads on write only
registers.
Can be implemented as a PAL equation *)

(* A function to read memory.  Will either  return the  data at a
RAM
location or the contents of a memory mapped register*)

(* invalid === the address is <=7 *)


fun valid_address a=
      (ram_address  a) |||
      (alu_address  a) |||
      (addressBit3  a);

exception can't_read;
```

```
        fun read state a=
              if ram_address a then get_mem state a
              else if addressBit4 a then
                Expand ( if addressBit1 a then
                 if addressBit0 a then get_carry (get_alu state)
                                  else get_overflow (get_alu state)
                  else if addressBit0 a then get_negative (get_alu state)

                       else get_zero ( get_alu state))
              else
if addressBit3 a then get_acc (get_alu state)
else raise can't_read;

        (* Write
              This function takes a  state, an  address and  a data
              item. It  will then  use this item to update the RISC
              state. The program counter will always be  adjusted -
              normally being  incremented, but after a write to the
              PC then it will be  changed  to  the  supplied value.
              Other internal registers - X,Y,skip,Acc and Carry can
              also  be  written  to.  Writing  to  the  other  ALU
              addresses causes  the ALU  to be activated to perform
              an operation upon the Accumulator,  the carry and the
              data supplied.
              Writes to  a ram  address cause the data to be stored
              at the specifed location. *)

        fun write state a data=
```

```
            if a=PC orelse (a=PLUS4 PC) then     (* program counter*)
                 ({       mem=get_mem state,
                          exstate={
                                  pc=Truncate15 data,
                                  x=get_x (get_ex state),
                                  y=get_y (get_ex state),
                                  halt=get_halt (get_ex state)},
                          alustate=get_alu state}:State)
            else
            if a=SKIP orelse (a=PLUS4 SKIP) then
                 {        mem=get_mem state,
                          exstate= {pc=
(if dataBit0 data then
S15 (get_pc (get_ex state))
else get_pc (get_ex state)),
                                  x=get_x (get_ex state),
                                  y=get_y (get_ex state),
                                  halt=get_halt (get_ex state)},
                          alustate=get_alu state}
            else
            if a=X orelse a=PLUS4 X then
                 {        mem=get_mem state,
                          exstate= {
                                  pc=get_pc (get_ex state),
                                  x=Truncate15 data,
                                  y=get_y (get_ex state),
                                  halt=get_halt (get_ex state)},
                          alustate=get_alu state}
            else
```

```
            if a=Y orelse a=PLUS4 Y then
                 {       mem=get_mem state,
                         exstate= {
                                 pc=get_pc (get_ex state),
                                 x=get_x (get_ex state),
                                 y=Truncate15 data,
                                 halt=get_halt (get_ex state)},
                         alustate=get_alu state}
            else
            if  ram_address a then
                 {       mem=RAM data (get_mem state) a,
                         exstate=get_ex state,
                         alustate=get_alu state}
            else
            if alu_address a then
                 {       mem = get_mem state,
                         exstate= get_ex state,
                         alustate= alu (get_alu state) data a
                         }
            else
                 (* else write to carry flag *)
                 (* warning: the state of the N,Z,V flags can't be predicted
                         after this operation: an ADD 0 operation should be
                         performed to reevaluate the other flags-
an action which'll clear the carry *)
                 {       mem = get_mem state,
exstate=get_ex state,
                         alustate= let
val { acc=_,z=z,n=n,v=v,carry=c}=
```

```
alu (get_alu state) data a
in
{ acc=get_acc (get_alu state),
                                        z=z,
                                        n=n,
                                        v=v,
                                        carry=c}
end
                        };
```

The description of the Execution Unit completes the specification of the Ultimate RISC. A function **execute** from **State** to **State** specifies and simulates the execution of a single instruction

```
(* execute.SIM v1.4
=========== *)

         (* simulation of the execution unit *)

           (* conditional index operation*)
          fun index a f i=if f then a else plus15 a i;

          fun execute (state:State)=
          let
                val {mem=m,
                     exstate={pc=pc,x=x,y=y,halt=halt},
                     alustate=a}=state
          in
                (* state if source fetch fails*)
```

```
            let val halted=({mem=m,
                        exstate={pc=S15 pc,
                                 x=x,
                                 y=y,
                                 halt=true},
                        alustate=a} :State)
                  (* state after the fetch of the first instruction---
                     the PC has been incremented *)
                and ss={mem=m,
                        exstate={pc=S15 pc,
                                 x=x,
y=y,
halt=false},
                        alustate=a}
            in
                  (* do nothing if already halted *)
                    if halt then state
                    (* fetch instruction*)
                    else if not (valid_address pc) then state
                    else let
                            val i=read state pc
                         in
                            (*fetch source operand*)
                            if not (valid_address (index (Source i)
      (IndexX i) x))
                            then halted
                            (*write to destination*)
                            else write ss
                               (index (Destination i) (IndexY i) y)
```

```
                                (read ss
                                     (index (Source i) (IndexX i) x))
                           end
                 end
          end;
```

To actually use the simulation some extra monitor functions were written. These provide memory reading and writing, code assembly, register access and all the functions one should expect from a monitor. The actual simulated computer is stored in a reference record, which the monitor functions address.

```
(* monitor.SIM v1.6  5/24/89
   ================ ======*)

          (* functions to control the simulation as the monitor should *)



          (* give starting state *)

          val reset_state=({mem=RAM0,
                         exstate={pc=Zero15,x=Zero15,
                                  y=Zero15,halt=false},
                         alustate={acc=Zero32,
                                   z=true,n=false,
                                   v=false,carry=false}}:State);

          (* describe machine state with a reference *)
```

```
  val URISC=ref reset_state;

          (* functions to display a state in an understandable form *)

          val print15=makestring o Int15toNat;
          val print32=makestring o Int32toNat;

          val CR="\n";

          fun dissassemble n=
                " MOVE"^
                (if IndexX n
                        then "X"
                        else "")^
                (if IndexY n then "Y " else " ")
                ^"("^
                (print15 (Source n))^ "," ^
                (print15 (Destination n))^
                 ")" ^ CR;



          fun show (state:State)=
          let val {mem=m,
                exstate={pc=pc,x=x,y=y,halt=halt},
                alustate={acc=acc,z=z,n=n,v=v,carry=c}}=state
          in
output(std_out,
                CR^"PC   ="^(print15 pc)^
```

```
                CR^"X    ="^(print15 x)^
                CR^"Y    ="^(print15 y)^
                CR^"halt ="^(makestring halt)^
                CR^"ACC  ="^(print32 acc)^
                CR^"(z,n,v,c)="^(makestring (z,n,v,c))^
                 (if not ( valid_address pc) then
                        (CR^"PC at write-only address"^CR)
                 else
                        (CR ^ "Current Instruction="^
                         (dissassemble (read state pc)))))


          end;

(* examine current registers *)
          fun ex ()=show (!URISC);

(* read an address *)
          fun r a =Int32toNat (read (!URISC)  (NattoInt15 a));

(* dump address count  - dumps memory locations *)

         fun dump _ 0 =()
           |  dump a n =
                (output(std_out,(CR^
                 (makestring a)^
                 "\t:\t"^
                 (makestring (r a))^
                 "\t=\t"^
```

```
                (dissassemble (read (!URISC) ( NattoInt15 a)))));
dump (a+1) (n-1));

          fun step ()=
                URISC:=execute (!URISC);

          (* step & show *)
          fun ss ()=(step();ex());

          fun run()=  (* step until halted *)
                if get_halt (get_ex  (!URISC)) then ()
                else (step();run());

          fun set_pc pc=
                URISC:={mem=get_mem (!URISC),
                        exstate={pc=( NattoInt15 pc),
                                 x=get_x (get_ex (!URISC)),
                                 y=get_y (get_ex (!URISC)),
                                 halt=get_halt ( get_ex (!URISC))},
                        alustate=get_alu (!URISC)};

          fun set_x x=
                URISC:={mem=get_mem (!URISC),
                        exstate={pc=get_pc ( get_ex (!URISC)),
                                 x=( NattoInt15 x),
                                 y=get_y ( get_ex (!URISC)),
                                 halt=get_halt ( get_ex (!URISC))},
                        alustate=get_alu (!URISC)};
```

```
fun set_y y=
      URISC:={mem=get_mem (!URISC),
              exstate={pc=get_pc (get_ex (!URISC)),
                       x=get_x ( get_ex (!URISC)),
                       y=( NattoInt15 y),
                       halt=get_halt ( get_ex (!URISC))},
              alustate=get_alu (!URISC)};

fun set_halt  halt=
      URISC:={mem=get_mem (!URISC),
              exstate={pc=get_pc ( get_ex (!URISC)),
                       x=get_x (get_ex (!URISC)),
                       y=get_y (get_ex (!URISC)),
                       halt=halt},
              alustate=get_alu (!URISC)};
fun set_acc a=
      URISC:={mem=get_mem (!URISC),
              exstate=get_ex (!URISC),
              alustate={acc=( NattoInt32 a),
                        z=get_zero ( get_alu (!URISC)),
                        n=get_negative ( get_alu (!URISC)),
                        v=get_overflow ( get_alu (!URISC)),
                        carry=get_carry ( get_alu (!URISC))}};
fun set_carry a=
      URISC:={mem=get_mem (!URISC),
              exstate=get_ex (!URISC),
              alustate={acc=get_acc (get_alu (!URISC)),
                        z=get_zero ( get_alu (!URISC)),
                        n=get_negative ( get_alu (!URISC)),
```

```
                                v=get_overflow ( get_alu (!URISC)),
                                carry=a}};


        (* go until halted *)
        fun go a=
              (set_pc  a;
               set_halt false;
               run();
               ex());

        (* write number to an address *)
        fun w address data=
              URISC:=write (!URISC) (NattoInt15 address) (NattoInt32 data);
        (* infix version of above *)

        infix 6 ##;
        fun a ## d=w a d;

val offset=65536;

        (* assemble a move instruction into memory *)
        infix 6 MOVE ;
        fun a MOVE (s ,d)=w a (offset*s +d);

        val INDEX=32768;

        infix 6 MOVEX ;
        fun a MOVEX (s, d)=a ## (offset*(INDEX+s)+d);
```

```
          infix 6 MOVEY ;
          fun a MOVEY (s ,d)=a ## (offset*s+d+INDEX);

          infix 6 MOVEXY ;
          fun a MOVEXY (s, d)=a ## (offset*(INDEX+s)+d+INDEX);

(* Now the Register Addresses in Integer Format *)
val PC=0;
val SKIP=1;
val X=2;
val Y=3;
val A=8;
val CIN=8;

(*The accumulator Functions *)

          val CLR= 16;
          val SUBA= 17;
          val SUBB= 18;
          val ADD= 19;
          val XOR= 20;
          val OR= 21;
          val AND= 22;
          val SET= 23;

(* extra offset to cause post-rotate *)
val SHIFT=8;
```

```
(* the condition codes *)
          val Z=CLR;
          val N=SUBA;
          val V=SUBB;
          val CARRY=ADD;




          (* can assemble  move instuctions e.g *)
          (* a short loop *)

          8192 MOVE (9000, CLR);
          8193 MOVE (9004 ,ADD);
          8194 MOVE (A, X);
          8195 MOVE (A, Y);
          8196 MOVEXY (9000, 10000);
          8197 MOVE (9001,CIN);
          8198 MOVE (9001, SUBB);
          8199 MOVE (N ,SKIP);
          8200 MOVE (9005, PC);
          8201 MOVE (0,0); (* halt *)
          9000  ## 0;
          9001  ## 1;
          9002  ## 2;
          9003  ## 3;
          9004  ## 4;
          9005  ## 8194;
```

Appendix G

Circuit Diagrams